Mechanized Specifications (2): WebAssembly and SpecTec

Sukyoung Ryu

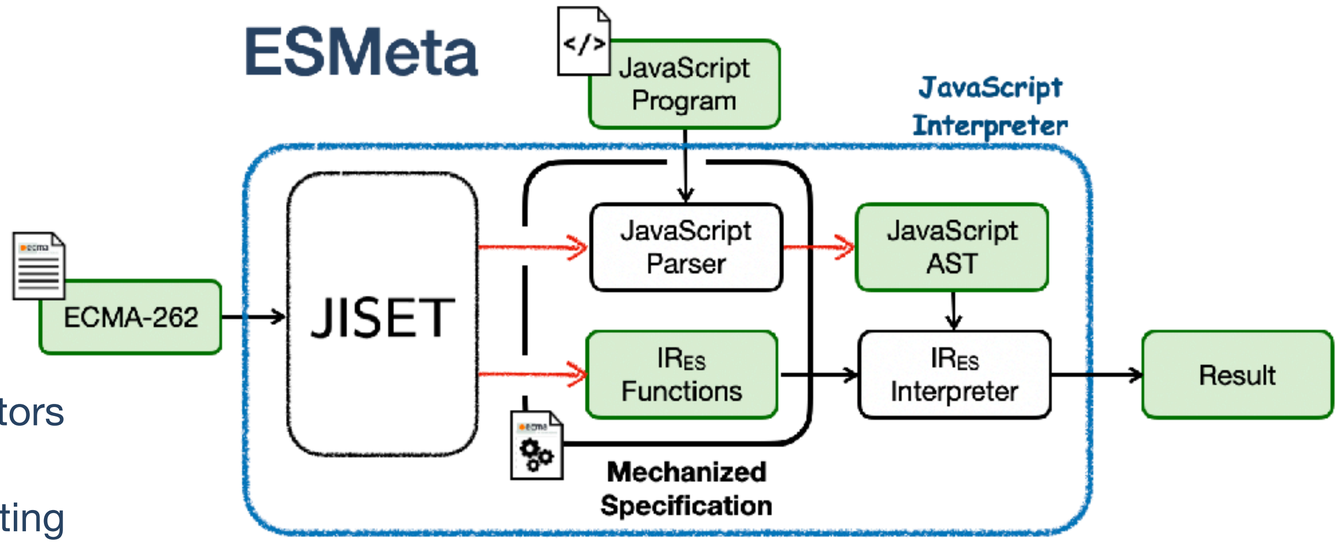
August 18, 2026

Mechanized Specifications

- **Lecture 1: JavaScript and ESMeta**
- **Lecture 2: WebAssembly and SpecTec**
- **Lecture 3: P4 and P4-SpecTec**
- **Lecture 4: Other PLs and Non-PL Software**

JavaScript and ESMeta

- **September 2020** JISET paper
- **May 2021** JEST paper
- **November 2021** JSTAR paper
- **November 2021** Meeting with the TC39 editors
- **January 2022** Presentation at the TC39 meeting
- **November 2022** ESMeta integrated into the CI of ECMA-262 and Test26
- **November 2022** JSAVER paper
- **June 2023** JEST_fs paper
- **May 2024** CACM paper



WebAssembly?

WebAssembly (Wasm) is a binary instruction format for a **stack-based** virtual machine, designed as a portable compilation target for programming languages.

t.binop

1. Assert: due to validation, two values of value type t are on the top of the stack.
2. Pop the value $t.\mathsf{const}\ c_2$ from the stack.
3. Pop the value $t.\mathsf{const}\ c_1$ from the stack.
4. If $binop_t(c_1, c_2)$ is defined, then:
   a. Let c be a possible result of computing $binop_t(c_1, c_2)$.
   b. Push the value $t.\mathsf{const}\ c$ to the stack.
5. Else:
   a. Trap.

$$
\begin{array}{lcll}
(t.\mathsf{const}\ c_1)\ (t.\mathsf{const}\ c_2)\ t.binop & \hookrightarrow & (t.\mathsf{const}\ c) & (\text{if } c \in binop_t(c_1, c_2)) \\
(t.\mathsf{const}\ c_1)\ (t.\mathsf{const}\ c_2)\ t.binop & \hookrightarrow & \mathsf{trap} & (\text{if } binop_t(c_1, c_2) = \{\})
\end{array}
$$

Dagstuhl Seminar 23101

Foundations of WebAssembly

(Mar 05 – Mar 10, 2023)

Permalink

Please use the following short url to reference this page: https://www.dagstuhl.de/23101

Organizers

- Karthikeyan Bhargavan (INRIA - Paris, FR)
- Jonathan Protzenko (Microsoft - Redmond, US)
- Andreas Rossberg (München, DE)
- Deian Stefan (University of California - San Diego, US)

Contact

- Andreas Dolzmann (for scientific matters)
- Christina Schwarz (for administrative matters)

timeline

2018 Wasm 1.0

focus on low-level languages to get off the ground quickly

2022 Wasm 2.0

multiple values, vector types, reference types, tabl

2023 Wasm 2.1

tail calls, relaxed vector ops, deterministic profile

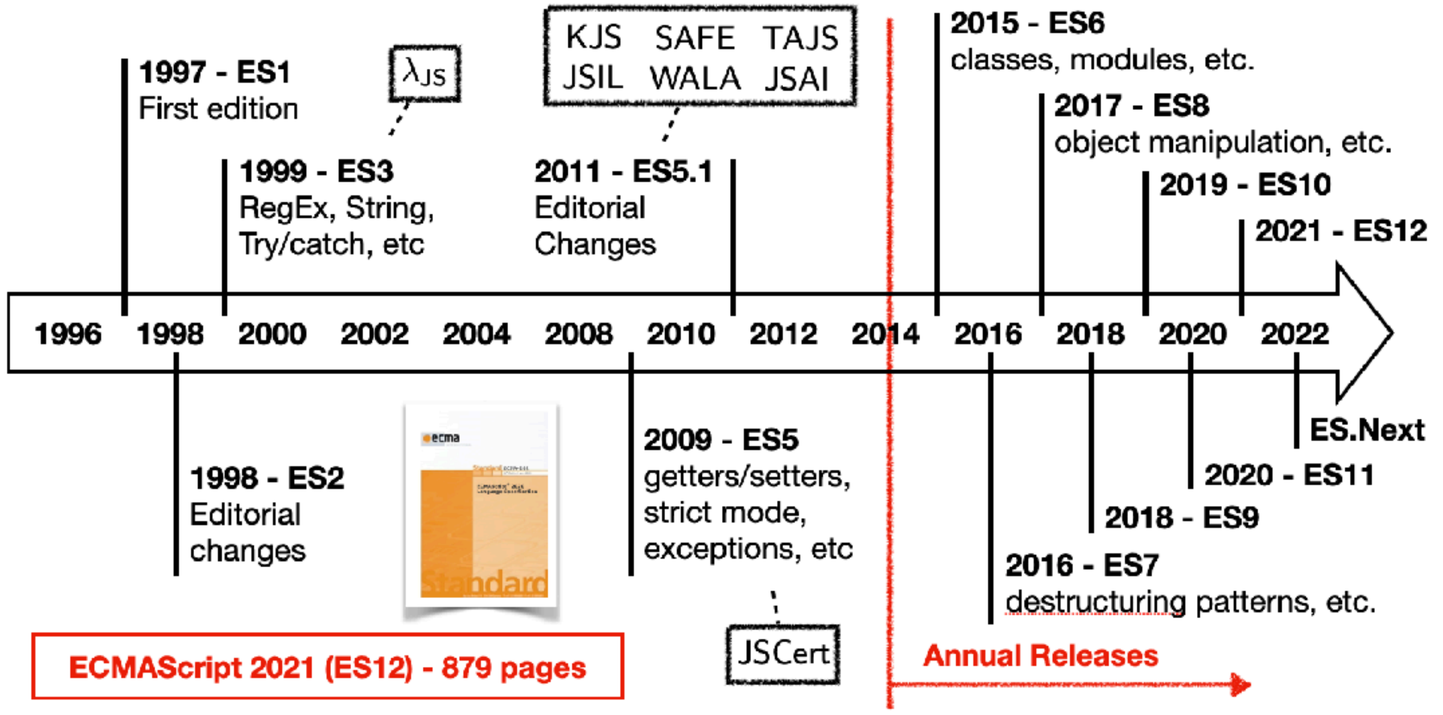
2024 Wasm 2.2+

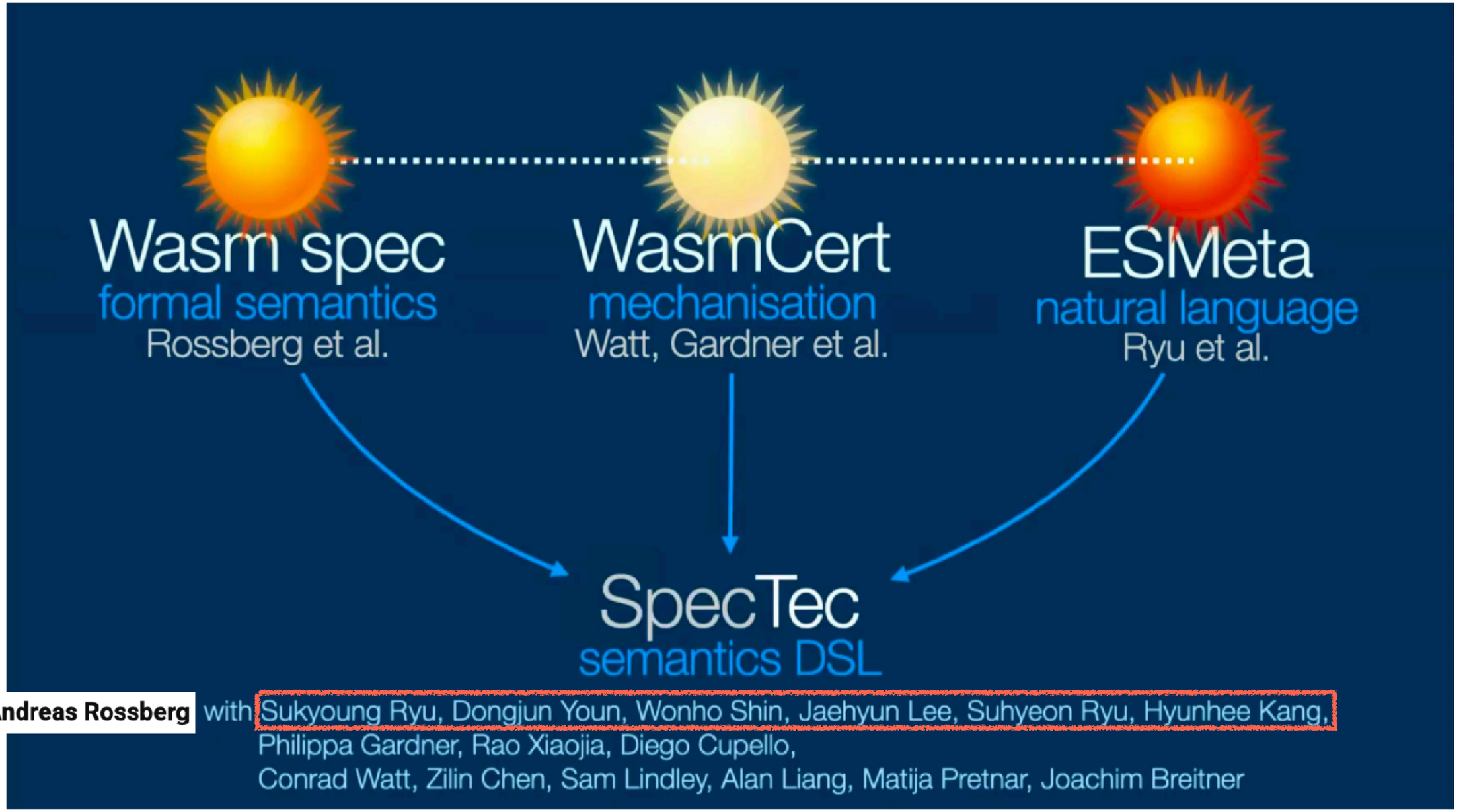
atomics, exceptions, typed references, garbage co

202X Wasm 2.X

stack switching, threading, memory control, type i

Problem: Fast Evolving JavaScript





https://www.youtube.com/watch?v=aAkhAaKlJIk

[PLDI 2017]

(store)	s	::=	$\{\text{inst } inst^*, \text{tab } tabinst^*, \text{mem } meminst^*\}$
(instances)	$inst$	::=	$\{\text{func } cl^*, \text{glob } v^*, \text{tab } i^?, \text{mem } i^?\}$
	$tabinst$	::=	cl^*
	$meminst$	::=	b^*
(closures)	cl	::=	$\{\text{inst } i, \text{ code } f\}$ (where f is not an import and has all exports ex^* erased)
(values)	v	::=	$t.\text{const } c$
(administrative operators)	e	::=	$\ldots \mid \textbf{trap} \mid \textbf{call } cl \mid \textbf{label}_n\{e^*\}\ e^*\ \textbf{end} \mid \textbf{local}_n\{i; v^*\}\ e^*\ \textbf{end}$
(local contexts)	L^0	::=	$v^*\ [\cdot]\ e^*$
	L^{k+1}	::=	$v^*\ \textbf{label}_n\{e^*\}\ L^k\ \textbf{end}\ e^*$

Reduction

$$\frac{s; v^*; e^* \hookrightarrow_i s'; v'^*; e'^*}{s; v^*; L^k[e^*] \hookrightarrow_i s'; v'^*; L^k[e'^*]} \qquad \frac{s; v^*; e^* \hookrightarrow_i s'; v'^*; e'^*}{s; v_0^*; \textbf{local}_n\{i; v^*\}\ e^*\ \textbf{end} \hookrightarrow_j s'; v_0^*; \textbf{local}_n\{i; v'^*\}\ e'^*\ \textbf{end}} \qquad \boxed{s; v^*; e^* \hookrightarrow_i s; v^*; e^*}$$

$$\begin{array}{rcll}
L^0[\textbf{trap}] & \hookrightarrow & \textbf{trap} & \text{if } L^0 \neq [\cdot] \\
(t.\text{const } c)\ t.unop & \hookrightarrow & t.\text{const } unop_t(c) & \\
(t.\text{const } c_1)\ (t.\text{const } c_2)\ t.binop & \hookrightarrow & t.\text{const } c & \text{if } c = binop_t(c_1, c_2) \\
(t.\text{const } c_1)\ (t.\text{const } c_2)\ t.binop & \hookrightarrow & \textbf{trap} & \text{otherwise} \\
(t.\text{const } c)\ t.testop & \hookrightarrow & \textbf{i32.const } testop_t(c) & \\
(t.\text{const } c_1)\ (t.\text{const } c_2)\ t.relop & \hookrightarrow & \textbf{i32.const } relop_t(c_1, c_2) & \\
(t_1.\text{const } c)\ t_2.\textbf{convert } t_1_sx^? & \hookrightarrow & t_2.\text{const } c' & \text{if } c' = \text{cvt}^{sx^?}_{t_1,t_2}(c) \\
(t_1.\text{const } c)\ t_2.\textbf{convert } t_1_sx^? & \hookrightarrow & \textbf{trap} & \text{otherwise} \\
(t_1.\text{const } c)\ t_2.\textbf{reinterpret } t_1 & \hookrightarrow & t_2.\text{const } \text{const}_{t_2}(\text{bits}_{t_1}(c)) & \\
\textbf{unreachable} & \hookrightarrow & \textbf{trap} & \\
\textbf{nop} & \hookrightarrow & \epsilon & \\
v\ \textbf{drop} & \hookrightarrow & \epsilon & \\
v_1\ v_2\ (\textbf{i32.const } 0)\ \textbf{select} & \hookrightarrow & v_2 & \\
v_1\ v_2\ (\textbf{i32.const } k+1)\ \textbf{select} & \hookrightarrow & v_1 & \\
v^n\ \textbf{block}\ (t_1^n \to t_2^m)\ e^*\ \textbf{end} & \hookrightarrow & \textbf{label}_m\{\varepsilon\}\ v^n\ e^*\ \textbf{end} & \\
v^n\ \textbf{loop}\ (t_1^n \to t_2^m)\ e^*\ \textbf{end} & \hookrightarrow & \textbf{label}_n\{\textbf{loop}\ (t_1^n \to t_2^m)\ e^*\ \textbf{end}\}\ v^n\ e^*\ \textbf{end} & \\
(\textbf{i32.const } 0)\ \textbf{if } tf\ e_1^*\ \textbf{else } e_2^*\ \textbf{end} & \hookrightarrow & \textbf{block } tf\ e_2^*\ \textbf{end} & \\
(\textbf{i32.const } k+1)\ \textbf{if } tf\ e_1^*\ \textbf{else } e_2^*\ \textbf{end} & \hookrightarrow & \textbf{block } tf\ e_1^*\ \textbf{end} & \\
\textbf{label}_n\{e^*\}\ v^*\ \textbf{end} & \hookrightarrow & v^* & \\
\textbf{label}_n\{e^*\}\ \textbf{trap end} & \hookrightarrow & \textbf{trap} & \\
\textbf{label}_n\{e^*\}\ L^j[v^n\ (\textbf{br } j)]\ \textbf{end} & \hookrightarrow & v^n\ e^* & \\
(\textbf{i32.const } 0)\ (\textbf{br_if } j) & \hookrightarrow & \epsilon & \\
(\textbf{i32.const } k+1)\ (\textbf{br_if } j) & \hookrightarrow & \textbf{br } j & \\
(\textbf{i32.const } k)\ (\textbf{br_table } j_1^k\ j\ j_2^*) & \hookrightarrow & \textbf{br } j & \\
(\textbf{i32.const } k+n)\ (\textbf{br_table } j_1^k\ j) & \hookrightarrow & \textbf{br } j & \\
s; \textbf{call } j & \hookrightarrow_i & \textbf{call } s_{\text{func}}(i, j) & \\
s; (\textbf{i32.const } j)\ \textbf{call_indirect } tf & \hookrightarrow_i & \textbf{call } s_{\text{tab}}(i, j) & \text{if } s_{\text{tab}}(i, j)_{\text{code}} = (\textbf{func } tf\ \textbf{local } t^*\ e^*) \\
s; (\textbf{i32.const } j)\ \textbf{call_indirect } tf & \hookrightarrow_i & \textbf{trap} & \text{otherwise} \\
v^n\ (\textbf{call } cl) & \hookrightarrow & \textbf{local}_m\{cl_{\text{inst}}; v^n\ (t.\text{const } 0)^k\}\ \textbf{block}\ (\epsilon \to t_2^m)\ e^*\ \textbf{end end} & \ldots \\
\textbf{local}_n\{i; v_i^*\}\ v^n\ \textbf{end} & \hookrightarrow & v^n & \mid \ \ldots \text{if } cl_{\text{code}} = (\textbf{func } (t_1^n \to t_2^m)\ \textbf{local } t^k\ e^*) \\
\textbf{local}_n\{i; v_i^*\}\ \textbf{trap end} & \hookrightarrow & \textbf{trap} & \\
\textbf{local}_n\{i; v_i^*\}\ L^k[v^n\ \textbf{return}]\ \textbf{end} & \hookrightarrow & v^n & \\
v_1^i\ v\ v_2^k; \textbf{get_local } j & \hookrightarrow & v & \\
v_1^j\ v\ v_2^k; v'\ (\textbf{set_local } j) & \hookrightarrow & v_1^j\ v'\ v_2^k; \epsilon & \\
v\ (\textbf{tee_local } j) & \hookrightarrow & v\ v\ (\textbf{set_local } j) & \\
s; \textbf{get_global } j & \hookrightarrow_i & s_{\text{glob}}(i, j) & \\
s; v\ (\textbf{set_global } j) & \hookrightarrow_i & s'; \epsilon & \text{if } s' = s \text{ with } \text{glob}(i, j) = v \\
s; (\textbf{i32.const } k)\ (t.\textbf{load } a\ o) & \hookrightarrow_i & t.\text{const } \text{const}_t(b^*) & \text{if } s_{\text{mem}}(i, k+o, |t|) = b^* \\
s; (\textbf{i32.const } k)\ (t.\textbf{load } tp_sx\ a\ o) & \hookrightarrow_i & t.\text{const } \text{const}^{sx}_t(b^*) & \text{if } s_{\text{mem}}(i, k+o, |tp|) = b^* \\
s; (\textbf{i32.const } k)\ (t.\textbf{load } tp_sx^?\ a\ o) & \hookrightarrow_i & \textbf{trap} & \text{otherwise} \\
s; (\textbf{i32.const } k)\ (\textbf{i32.const } c)\ (t.\textbf{store } a\ o) & \hookrightarrow_i & s'; \epsilon & \text{if } s' = s \text{ with } \text{mem}(i, k+o, |t|) = \text{bits}_t^{|t|}(c) \\
s; (\textbf{i32.const } k)\ (t.\text{const } c)\ (t.\textbf{store } tp\ a\ o) & \hookrightarrow_i & s'; \epsilon & \text{if } s' = s \text{ with } \text{mem}(i, k+o, |tp|) = \text{bits}_t^{|tp|}(c) \\
s; (\textbf{i32.const } k)\ (t.\text{const } c)\ (t.\textbf{store } tp^?\ a\ o) & \hookrightarrow_i & \textbf{trap} & \text{otherwise} \\
s; \textbf{current_memory} & \hookrightarrow_i & \textbf{i32.const } |s_{\text{mem}}(i, *)|/64\,\text{Ki} & \\
s; (\textbf{i32.const } k)\ \textbf{grow_memory} & \hookrightarrow_i & s'; \textbf{i32.const } |s_{\text{mem}}(i, *)|/64\,\text{Ki} & \text{if } s' = s \text{ with } \text{mem}(i, *) = s_{\text{mem}}(i, *)\ (0)^{k \cdot 64\,\text{Ki}} \\
s; (\textbf{i32.const } k)\ \textbf{grow_memory} & \hookrightarrow_i & \textbf{i32.const } (-1) &
\end{array}$$

Figure 2. Small-step reduction rules

(contexts) $C ::= \{\text{func } tf^*, \text{global } tg^*, \text{table } n^?, \text{memory } n^?, \text{local } t^*, \text{label } (t^*)^*, \text{return } (t^*)^?\}$

Typing Instructions $\boxed{C \vdash e^* : tf}$

$$\frac{}{C \vdash t.\textbf{const } c : \epsilon \to t} \quad \frac{}{C \vdash t.unop : t \to t} \quad \frac{}{C \vdash t.binop : t\ t \to t} \quad \frac{}{C \vdash t.testop : t \to \textbf{i32}} \quad \frac{}{C \vdash t.relop : t\ t \to \textbf{i32}}$$

$$\frac{t_1 \neq t_2 \quad sx^? = \epsilon \Leftrightarrow (t_1 = \textbf{in} \wedge t_2 = \textbf{in}' \wedge |t_1| < |t_2|) \vee (t_1 = \textbf{fn} \wedge t_2 = \textbf{fn}')}{C \vdash t_1.\textbf{convert } t_2\_sx^? : t_2 \to t_1} \quad \frac{t_1 \neq t_2 \quad |t_1| = |t_2|}{C \vdash t_1.\textbf{reinterpret } t_2 : t_2 \to t_1}$$

$$\frac{}{C \vdash \textbf{unreachable} : t_1^* \to t_2^*} \quad \frac{}{C \vdash \textbf{nop} : \epsilon \to \epsilon} \quad \frac{}{C \vdash \textbf{drop} : t \to \epsilon} \quad \frac{}{C \vdash \textbf{select} : t\ t\ \textbf{i32} \to t}$$

$$\frac{tf = t_1^n \to t_2^m \quad C, \text{label } (t_2^m) \vdash e^* : tf}{C \vdash \textbf{block } tf\ e^*\ \textbf{end} : tf} \quad \frac{tf = t_1^n \to t_2^m \quad C, \text{label } (t_1^n) \vdash e^* : tf}{C \vdash \textbf{loop } tf\ e^*\ \textbf{end} : tf}$$

$$\frac{tf = t_1^n \to t_2^m \quad C, \text{label } (t_2^m) \vdash e_1^* : tf \quad C, \text{label } (t_2^m) \vdash e_2^* : tf}{C \vdash \textbf{if } tf\ e_1^*\ \textbf{else } e_2^*\ \textbf{end} : t_1^n\ \textbf{i32} \to t_2^m}$$

$$\frac{C_{\text{label}}(i) = t^*}{C \vdash \textbf{br } i : t_1^*\ t^* \to t_2^*} \quad \frac{C_{\text{label}}(i) = t^*}{C \vdash \textbf{br\_if } i : t^*\ \textbf{i32} \to t^*} \quad \frac{(C_{\text{label}}(i) = t^*)^+}{C \vdash \textbf{br\_table } i^+ : t_1^*\ t^*\ \textbf{i32} \to t_2^*}$$

$$\frac{C_{\text{return}} = t^*}{C \vdash \textbf{return} : t_1^*\ t^* \to t_2^*} \quad \frac{C_{\text{func}}(i) = tf}{C \vdash \textbf{call } i : tf} \quad \frac{tf = t_1^* \to t_2^* \quad C_{\text{table}} = n}{C \vdash \textbf{call\_indirect } tf : t_1^*\ \textbf{i32} \to t_2^*}$$

$$\frac{C_{\text{local}}(i) = t}{C \vdash \textbf{get\_local } i : \epsilon \to t} \quad \frac{C_{\text{local}}(i) = t}{C \vdash \textbf{set\_local } i : t \to \epsilon} \quad \frac{C_{\text{local}}(i) = t}{C \vdash \textbf{tee\_local } i : t \to t} \quad \frac{C_{\text{global}}(i) = \text{mut}^? t}{C \vdash \textbf{get\_global } i : \epsilon \to t} \quad \frac{C_{\text{global}}(i) = \textbf{mut } t}{C \vdash \textbf{set\_global } i : t \to \epsilon}$$

$$\frac{C_{\text{memory}} = n \quad 2^a \leq (|tp| <)^?|t| \quad (tp\_sz)^? = \epsilon \vee t = \textbf{im}}{C \vdash t.\textbf{load } (tp\_sz)^?\ a\ o : \textbf{i32} \to t} \quad \frac{C_{\text{memory}} = n \quad 2^a \leq (|tp| <)^?|t| \quad tp^? = \epsilon \vee t = \textbf{im}}{C \vdash t.\textbf{store } tp^?\ a\ o : \textbf{i32}\ t \to \epsilon}$$

$$\frac{C_{\text{memory}} = n}{C \vdash \textbf{current\_memory} : \epsilon \to \textbf{i32}} \quad \frac{C_{\text{memory}} = n}{C \vdash \textbf{grow\_memory} : \textbf{i32} \to \textbf{i32}}$$

$$\frac{}{C \vdash \epsilon : \epsilon \to \epsilon} \quad \frac{C \vdash e_1^* : t_1^* \to t_2^* \quad C \vdash e_2 : t_2^* \to t_3^*}{C \vdash e_1^*\ e_2 : t_1^* \to t_3^*} \quad \frac{C \vdash e^* : t_1^* \to t_2^*}{C \vdash e^* : t^*\ t_1^* \to t^*\ t_2^*}$$

Typing Modules

$$\frac{tf = t_1^* \to t_2^* \quad C, \text{local } t_1^*\ t^*, \text{label } (t_2^*), \text{return } (t_2^*) \vdash e^* : \epsilon \to t_2^*}{C \vdash ex^*\ \textbf{func } tf\ \textbf{local } t^*\ e^* : ex^*\ tf} \quad \frac{tg = \text{mut}^?\ t \quad C \vdash e^* : \epsilon \to t \quad ex^* = \epsilon \vee tg = t}{C \vdash ex^*\ \textbf{global } tg\ e^* : ex^*\ tg}$$

$$\frac{(C_{\text{func}}(i) = tf)^n}{C \vdash ex^*\ \textbf{table } n\ i^n : ex^*\ n} \quad \frac{}{C \vdash ex^*\ \textbf{memory } n : ex^*\ n}$$

$$\frac{tg = t}{C \vdash ex^*\ \textbf{func } tf\ im : ex^*\ tf} \quad \frac{}{C \vdash ex^*\ \textbf{global } tg\ im : ex^*\ tg} \quad \frac{}{C \vdash ex^*\ \textbf{table } n\ im : ex^*\ n} \quad \frac{}{C \vdash ex^*\ \textbf{memory } n\ im : ex^*\ n}$$

$$\frac{(C \vdash f : ex_f^*\ tf)^*\ \ (C_i \vdash glob_i : ex_g^*\ tg_i)_i^*\ \ (C \vdash tab : ex_t^*\ n)^?\ \ (C - mem : ex_m^*\ n)^*\ \ (C_i = \{\text{global } tg^{(i-1)}\})_i^*\ \ C = \{\text{func } tf^*, \text{global } tg^*, \text{table } n^?, \text{memory } n^?\}\ \ ex_f^{**}\ ex_g^{**}\ ex_t^{*?}\ ex_m^{*?} \text{ distinct}}{\vdash \textbf{module } f^*\ glob^*\ tab^?\ mem^?}$$

Figure 3. Typing rules

4.2 Soundness

The WebAssembly type system enjoys standard *soundness* properties [41]. Soundness proves that the reduction rules from Section 3 actually cover all execution states that can arise for valid programs. In other words, it proves the absence of undefined behavior in the execution semantics (assuming the auxiliary numeric primitives are well-defined).

In particular, this implies the absence of *type safety* violations such as invalid calls or illegal accesses to locals, it guarantees *memory safety*, and it ensures the inaccessibility of code addresses or the call stack. It also implies that the use of the operand stack is structured and its layout determined statically at all program points, which is crucial for efficient compilation on a register machine. Furthermore, it establishes memory and state *encapsulation* – i.e., abstraction properties on the module and function boundaries, which cannot leak information unless explicitly exported/returned – necessary conditions for user-defined security measures.

Store Typing Before we can state the soundness theorems concretely, we must extend the typing rules to stores and configurations as defined in Figure 2. These rules, shown in Figure 4, are not required for validation, but for generalizing to dynamic computations. They use an additional *store context* S to classify the store. The typing judgement for instructions in Figure 3 is extended to $S; C \vdash e^* : tf$ by implicitly adding S to all rules – S is never modified or used by those rules, but is accessed by the new rules for **call** cl and **local**.

Wasm Is …

- Large
  - over 500 instructions in Wasm 3.0
  - non-trivial type system
- Growing fast
  - over 20 extensions since 1.0
  - still missing important features
  - (currently) 15 completed and 20+ active proposals

Challenges

- Prose is a hand-crafted translation of formal rules
  - laborious, tedious, error-prone
  - for political reasons, written in markup (reStructuredText) + tooling hacks
- Code reviews are painful and unreliable
  - spec diffs for a single proposal can be >20,000 LOC!
- Yet more work created for mechanizations
  - difficult to keep up

Deliverables for a Wasm Proposal

1. Updated formal rules (LaTeX)
2. Updated prose description (reST markdown)
3. Updated reference interpreter (OCaml)
4. Updated test suite (Wasm)

Not yet required, but somebody eventually has to do it:

5. Updated mechanized definitions (Rocq / Isabelle)
6. Updated mechanized proofs (Rocq / Isabelle)

select $(t^*)^?$

1. Assert: due to validation, a value of value type i32 is on the top of the stack.
2. Pop the value i32.const c from the stack.
3. Assert: due to validation, two more values (of the same value type) are on the top of the stack.
4. Pop the value val_2 from the stack.
5. Pop the value val_1 from the stack.
6. If c is not 0, then:
   a. Push the value val_1 back to the stack.
7. Else:
   a. Push the value val_2 back to the stack.

$$val_1\ val_2\ (\text{i32.const}\ c)\ (\text{select}\ t^?) \hookrightarrow val_1 \quad (\text{if}\ c \neq 0)$$
$$val_1\ val_2\ (\text{i32.const}\ c)\ (\text{select}\ t^?) \hookrightarrow val_2 \quad (\text{if}\ c = 0)$$

```
rule Step_pure/select-true:
  val_1 val_2 (CONST I32 c) (SELECT t*?)  ~>  val_1
  -- if c =/= 0

rule Step_pure/select-false:
  val_1 val_2 (CONST I32 c) (SELECT t*?)  ~>  val_2
  -- if c = 0
```

```
.. _exec-select:

:math:`\SELECT~(t^\ast)^?`
..........................

1. Assert: due to :ref:`validation <valid-select>`, a :ref:`value <
syntax-value>` of :ref:`value type <syntax-valtype>` |I32| is on
the top of the :ref:`stack <syntax-stack>`.

2. Pop the value :math:`\I32.\CONST~c` from the :ref:`stack <
syntax-stack>`.

3. Assert: due to :ref:`validation <valid-select>`, two more :ref:`
values <syntax-value>` (of the same :ref:`value type <
syntax-valtype>`) are on the top of the stack.

4. Pop the :ref:`value <syntax-value>` :math:`\val_2` from the :ref:
`stack <syntax-stack>`.

5. Pop the :ref:`value <syntax-value>` :math:`\val_1` from the :ref:
`stack <syntax-stack>`.

6. If :math:`c` is not :math:`0`, then:

   a. Push the :ref:`value <syntax-value>` :math:`\val_1` back to
   the :ref:`stack <syntax-stack>`.

7. Else:

   a. Push the :ref:`value <syntax-value>` :math:`\val_2` back to
   the :ref:`stack <syntax-stack>`.

.. math::
   \begin{array}{lcl@{\qquad}l}
   \val_1~\val_2~(\I32\K{.}\CONST~c)~(\SELECT~t^?) &\stepto& \val_1
     & (\iff c \neq 0) \\
   \val_1~\val_2~(\I32\K{.}\CONST~c)~(\SELECT~t^?) &\stepto& \val_2
     & (\iff c = 0) \\
   \end{array}
```

```
.. index:: heap type, type identifier
   pair: validation; heap type
   single: abstract syntax; heap type
.. _valid-heaptype:

Heap Types
~~~~~~~~~~

Concrete :ref:`Heap types <syntax-heaptype>` are only valid when
the :ref:`type index <syntax-typeidx>` is.

:math:`\absheaptype`
....................

* The heap type is valid.

.. math::
   \frac{
   }{
     C \vdashheaptype \absheaptype \ok
   }

:math:`\typeidx`
................

* The type :math:`C.\CTYPES[\typeidx]` must be defined in the context.

* Then the heap type is valid.

.. math::
   \frac{
     C.\CTYPES[\typeidx] = \deftype
   }{
     C \vdashheaptype \typeidx \ok
   }


.. index:: reference type, heap type
   pair: validation; reference type
   single: abstract syntax; reference type
.. _valid-reftype:

Reference Types
~~~~~~~~~~~~~~~

:ref:`Reference types <syntax-reftype>` are valid when the referenced
:ref:`heap type <syntax-heaptype>` is.

:math:`\REF~\NULL^?~\heaptype`
..............................

* The heap type :math:`\heaptype` must be :ref:`valid <valid-heaptype>`.

* Then the reference type is valid.

.. math::
   \frac{
     C \vdashreftype \heaptype \ok
   }{
     C \vdashreftype \REF~\NULL^?~\heaptype \ok
   }
```

```
Heap Types
~~~~~~~~~~

Concrete :ref:`Heap types <syntax-heaptype>` are only valid when
the :ref:`type index <syntax-typeidx>` is.

${prose: Heaptype_ok/absheaptype}

${rule: Heaptype_ok/absheaptype}

${prose: Heaptype_ok/typeidx}

${rule: Heaptype_ok/typeidx)

Reference Types
~~~~~~~~~~~~~~~

:ref:`Reference types <syntax-reftype>` are valid when the referenced
:ref:`heap type <syntax-heaptype>` is.

${prose: Reftype_ok}

${rule: Reftype_ok}
```

br l

1. Assert: due to validation, the stack contains at least $l + 1$ labels.
2. Let L be the l-th label appearing on the stack, starting from the top and counting from zero.
3. Let n be the arity of L.
4. Assert: due to validation, there are at least n values on the top of the stack.
5. Pop the values val^n from the stack.
6. Repeat $l + 1$ times:
   a. While the top of the stack is a value, do:
      i. Pop the value from the stack.
   b. Assert: due to validation, the top of the stack now is a label.
   c. Pop the label from the stack.
7. Push the values val^n to the stack.
8. Jump to the continuation of L.

$$\mathsf{label}_n\{instr^*\}\ B^l[val^n\ (\mathsf{br}\ l)]\ \mathsf{end} \quad \hookrightarrow \quad val^n\ instr^*$$

br_if l

1. Assert: due to validation, a value of value type i32 is on the top of the stack.
2. Pop the value i32.const c from the stack.
3. If c is non-zero, then:
   a. Execute the instruction $(\mathsf{br}\ l)$.
4. Else:
   a. Do nothing.

$$\begin{array}{lcll} (\mathsf{i32.const}\ c)\ (\mathsf{br\_if}\ l) & \hookrightarrow & (\mathsf{br}\ l) & (\mathsf{if}\ c \neq 0) \\ (\mathsf{i32.const}\ c)\ (\mathsf{br\_if}\ l) & \hookrightarrow & \epsilon & (\mathsf{if}\ c = 0) \end{array}$$

br_table $l^*\ l_N$

1. Assert: due to validation, a value of value type i32 is on the top of the stack.
2. Pop the value i32.const i from the stack.
3. If i is smaller than the length of l^*, then:
   a. Let l_i be the label $l^*[i]$.
   b. Execute the instruction $(\mathsf{br}\ l_i)$.
4. Else:
   a. Execute the instruction $(\mathsf{br}\ l_N)$.

$$\begin{array}{lcll} (\mathsf{i32.const}\ i)\ (\mathsf{br\_table}\ l^*\ l_N) & \hookrightarrow & (\mathsf{br}\ l_i) & (\mathsf{if}\ l^*[i] = l_i) \\ (\mathsf{i32.const}\ i)\ (\mathsf{br\_table}\ l^*\ l_N) & \hookrightarrow & (\mathsf{br}\ l_N) & (\mathsf{if}\ |l^*| \leq i) \end{array}$$

br l

1. If the first non-value entry of the stack is a label, then:
   a. Let L be the topmost label.
   b. Let n be the arity of L
   c. If $l = 0$, then:
      1) Assert: Due to validation, there are at least n values on the top of the stack.
      2) Pop the values val^n from the stack.
      3) Pop all values val'^* from the top of the stack.
      4) Pop the label L from the stack.
      5) Push the values val^n to the stack.
      6) Jump to the continuation of L.
   d. Else:
      1) Pop all values val^* from the top of the stack.
      2) If $l > 0$, then:
         a) Pop the label L from the stack.
         b) Push the values val^* to the stack.
         c) Execute the instruction $(\mathsf{br}\ l - 1)$.
2. Else if the first non-value entry of the stack is a handler, then:
   a. Pop all values val^* from the top of the stack.
   b. Pop the handler H from the stack.
   c. Push the values val^n to the stack.
   d. Execute the instruction $(\mathsf{br}\ l)$.

$$\begin{array}{rcll} (\mathsf{label}_n\{instr'^*\}\ val'^*\ val^n\ (\mathsf{br}\ l)\ instr^*) & \hookrightarrow & val^n\ instr'^* & \text{if } l = 0 \\ (\mathsf{label}_n\{instr'^*\}\ val'^*\ (\mathsf{br}\ l)\ instr^*) & \hookrightarrow & val^*\ (\mathsf{br}\ l - 1) & \text{if } l > 0 \end{array}$$

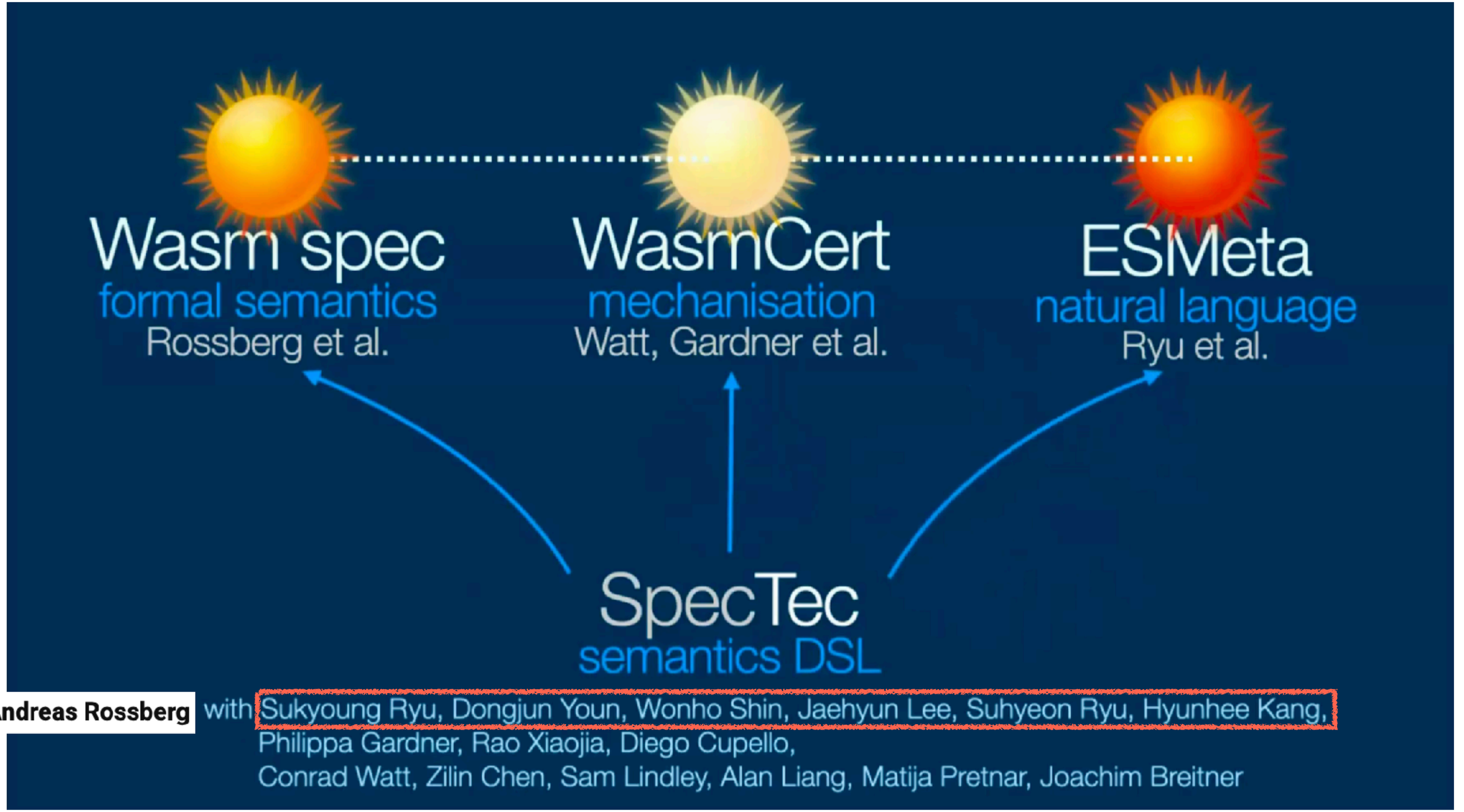
br_if l

1. Assert: Due to validation, a value of number type i32 is on the top of the stack.
2. Pop the value $(\mathsf{i32.const}\ c)$ from the stack.
3. If $c \neq 0$, then:
   a. Execute the instruction $(\mathsf{br}\ l)$.
4. Else:
   a. Do nothing.

$$\begin{array}{lcll} (\mathsf{i32.const}\ c)\ (\mathsf{br\_if}\ l) & \hookrightarrow & (\mathsf{br}\ l) & \text{if } c \neq 0 \\ (\mathsf{i32.const}\ c)\ (\mathsf{br\_if}\ l) & \hookrightarrow & \epsilon & \text{if } c = 0 \end{array}$$

br_table $l^*\ l'$

1. Assert: Due to validation, a value of number type i32 is on the top of the stack.
2. Pop the value $(\mathsf{i32.const}\ i)$ from the stack.
3. If $i < |l^*|$, then:
   a. Execute the instruction $(\mathsf{br}\ l^*[i])$.
4. Else:



https://www.youtube.com/watch?v=aAkhAaKlJIk

Thu 27 Jun 2024 11:20 - 11:40 at Iceland / Denmark - Managed Languages

☆ Bringing the WebAssembly Standard up to Speed with SpecTec

WebAssembly (Wasm) is a portable low-level bytecode language and virtual machine that has seen increasing use in a variety of ecosystems. Its specification is unusually rigorous — including a full formal semantics for the language — and every new feature must be specified in this formal semantics, in prose, and in the official reference interpreter before it can be standardized. With the growing size of the language, this manual process with its redundancies has become laborious and error-prone, and in this work, we offer a solution. We present SpecTec, a domain-specific language (DSL) and toolchain that facilitates both the Wasm specification and the generation of artifacts necessary to standardize new features. SpecTec serves as a single source of truth — from a SpecTec definition of the Wasm semantics, we can generate a typeset specification, including formal definitions and prose pseudocode descriptions, and a meta-level interpreter. Further backends for test generation and interactive theorem proving are planned. We evaluate SpecTec's ability to represent the latest Wasm 2.0 and show that the generated meta-level interpreter passes 100% of the applicable official test suite. We show that SpecTec is highly effective at discovering and preventing errors by detecting historical errors in the specification that have been corrected and ten errors in five proposals ready for inclusion in the next version of Wasm. Our ultimate aim is that SpecTec should be adopted by the Wasm standards community and used to specify future versions of the standard.

DOI

Dongjun Youn
KAIST
South Korea

Jaehyun Lee
KAIST

Joachim Breitner
unaffiliated
Germany

Sam Lindley
University of Edinburgh
United Kingdom

Xiaojia Rao
Imperial College
United Kingdom

Andreas Rossberg
Independent
Germany

Shin Wonho
KAIST

Sukyoung Ryu
KAIST
South Korea

Philippa Gardner
Imperial College London
United Kingdom

Matija Pretnar
University of Ljubljana
Slovenia

Conrad Watt
Nanyang Technological University
Singapore

Bringing the WebAssembly Standard up to Speed with SpecTec

Dongjun Youn, Wonho Shin, Jaehyun Lee, Sukyoung Ryu, Joachim Breitner, Philippa Gardner, Sam Lindley, Matija Pretnar, Xiaojia Rao, Conrad Watt, Andreas Rossberg

2024-06-27

PLDI COPENHAGEN 2024

WebAssembly (Wasm)

WebAssembly is a low-level code format
designed for efficient execution
especially on the web

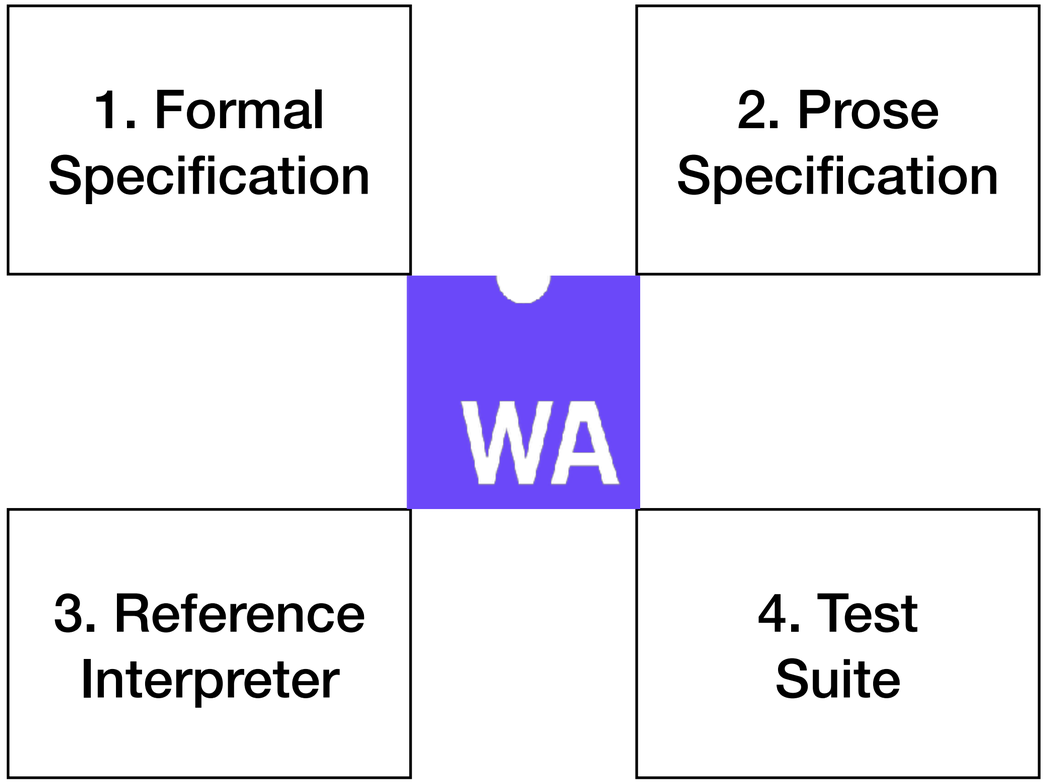
```
int f(int x, int y) {
  return x + y;
}
```

```
(func (export "f")
  (param i32 i32) (result i32)
  local.get 0 ;; [ x ]
  local.get 1 ;; [ x , y ]
  i32.add     ;; [ x + y ]
)
```

https://webassembly.github.io/spec/core/intro/introduction.html

WebAssembly Artifacts

To mitigate the risk of implementation divergence,
Wasm requires four artifacts for a new feature to be standardized



WebAssembly Artifacts

To mitigate the risk of implementation divergence,
Wasm requires four artifacts for a new feature to be standardized

$$(t.\mathsf{const}\ c_1)\ (t.\mathsf{const}\ c_2)\ t.\mathit{relop} \quad \hookrightarrow \quad (\mathsf{i32.const}\ c)$$
$$(\text{if } c = \mathit{relop}_t(c_1, c_2))$$

<1. Formal Specification (LaTeX)>

t.relop

1. Assert: Two values of value type t are on the top of the stack.
2. Pop the value $t.\mathsf{const}\ c_2$ from the stack.
3. Pop the value $t.\mathsf{const}\ c_1$ from the stack.
4. Let c be the result of computing $\mathit{relop}_t(c_1, c_2)$.
5. Push the value $\mathsf{i32.const}\ c$ to the stack.

<2. Prose Specification (reST)>

WebAssembly Artifacts

To mitigate the risk of implementation divergence,
Wasm requires four artifacts for a new feature to be standardized

```
(match e', vs with
| Unreachable, vs ->
  vs, [Trapping "unreachable executed" @@ e.at]

| Nop, vs ->
  vs, []
```

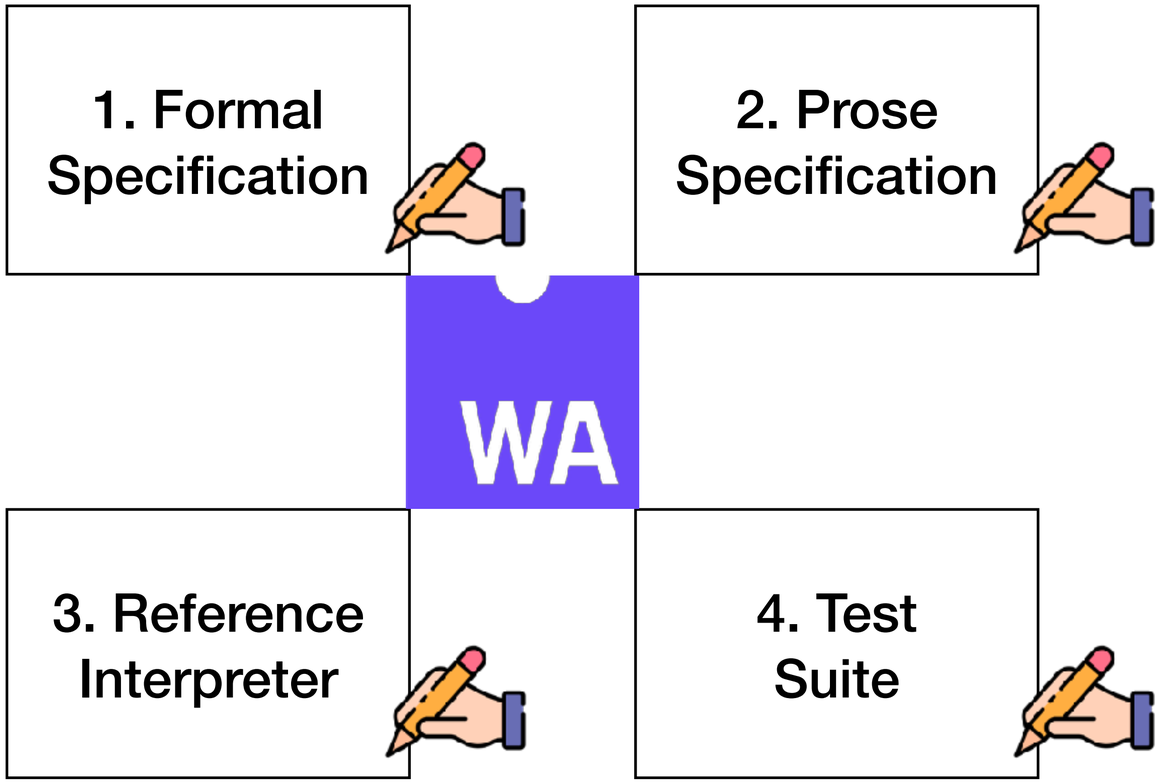
<3. Reference Interpreter (OCaml)>

```
(assert_return (invoke "fac" (i64.const 0)) (i64.const 1))
(assert_return (invoke "fac" (i64.const 1)) (i64.const 1))
(assert_return (invoke "fac" (i64.const 5)) (i64.const 120))
```

<4. Test Suite (wast)>

WebAssembly Artifacts - Limitations

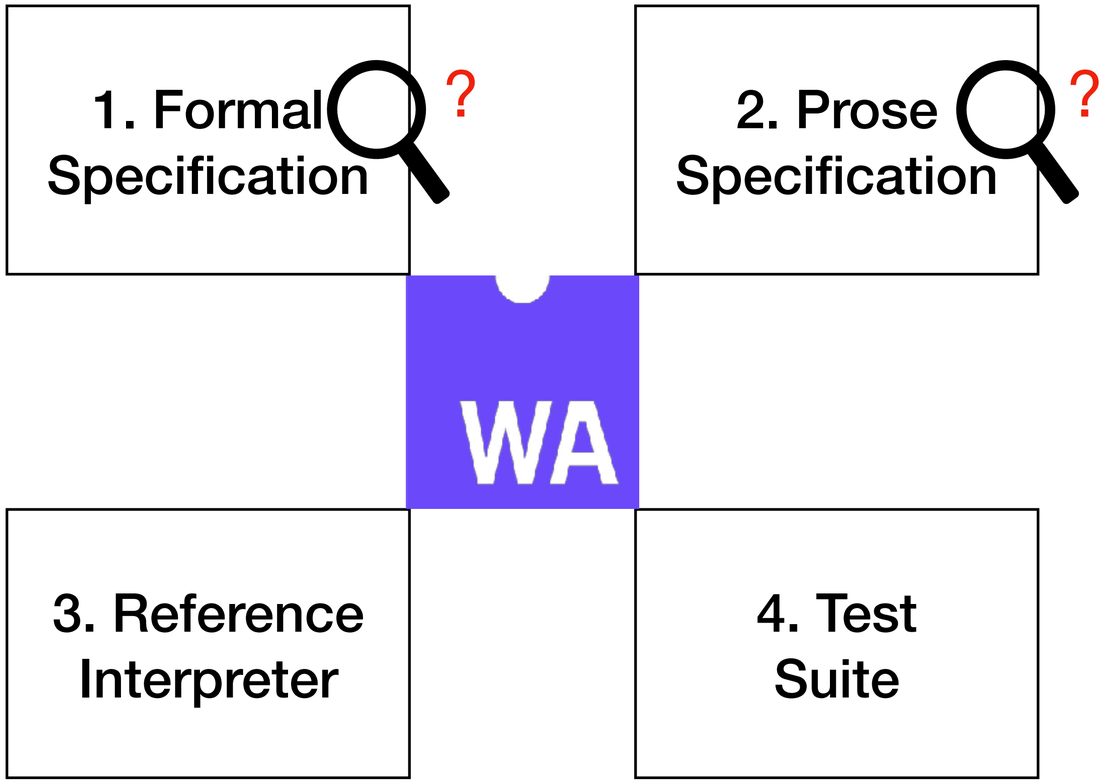
Laborious:
- Manual documentation & maintenance



WebAssembly Artifacts - Limitations

Laborious:
- Manual documentation & maintenance

Bad readability:
- Specs written in LaTeX / reST
- Code reviews are not user-friendly

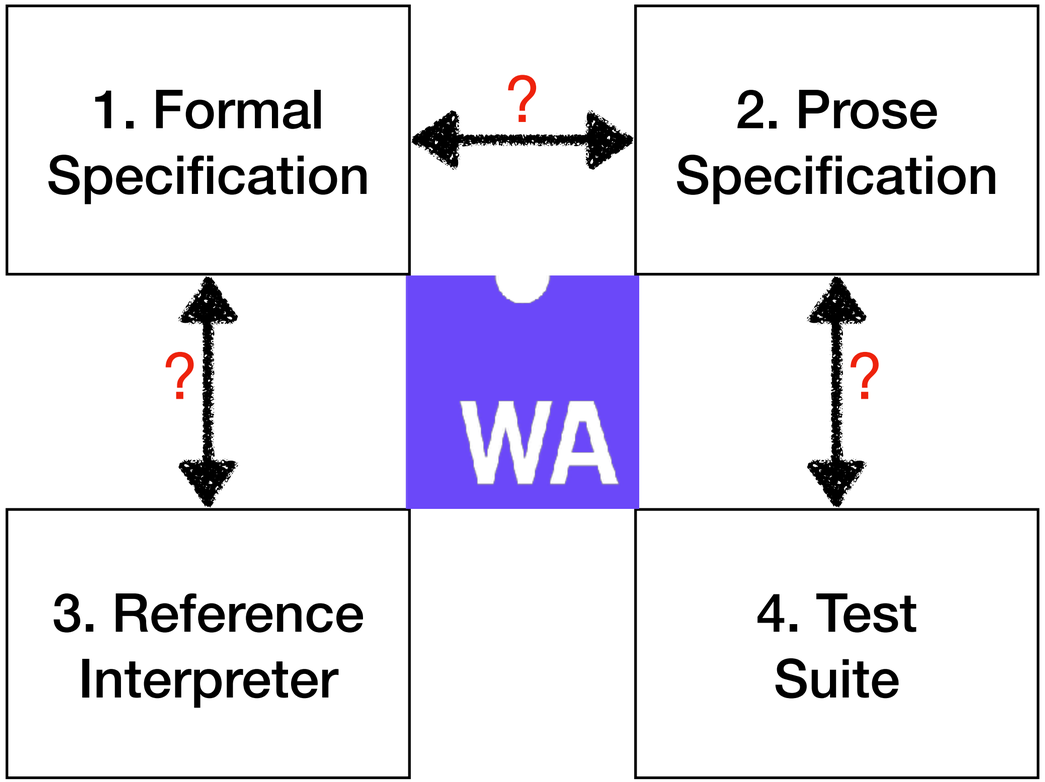


WebAssembly Artifacts - Limitations

Laborious:
- Manual documentation & maintenance

Bad readability:
- Specs written in LaTeX / reST
- Code reviews are not user-friendly

Error-Prone:
- Correctness / consistency
  rely on eye-ball correspondence



WebAssembly Artifacts - Limitations

```
.. _exec-relop:

:math:`{\mathit{nt}} . {\mathit{relop}}`
.....................................
1. Assert: Due to validation, a value of value type :math:`{\mathit{nt}}` is on the top of the stack.
2. Pop the value :math:`({\mathit{nt}}{.}\mathsf{const}~c_2)` from the stack.
3. Assert: Due to validation, a value of value type :math:`{\mathit{nt}}` is on the top of the stack.
4. Pop the value :math:`({\mathit{nt}}{.}\mathsf{const}~c_1)` from the stack.
5. Let :math:`c` be :math:`{{{\mathit{relop}}}{}_{{\mathit{nt}}}}{(c_1,\, c_2)}`.
6. Push the value :math:`(\mathsf{i{\scriptstyle32}}{.}\mathsf{const}~c)` to the stack.

\

.. math::
   \begin{array}{@{}l@{}rcl@{}l@{}}
   & ({\mathit{nt}}{.}\mathsf{const}~c_1)~({\mathit{nt}}{.}\mathsf{const}~c_2)~({\mathit{nt}} .
{\mathit{relop}}) &\hookrightarrow& (\mathsf{i{\scriptstyle32}}{.}\mathsf{const}~c)
     &\qquad \mbox{if}~c = {{{\mathit{relop}}}{}_{{\mathit{nt}}}}{(c_1,\, c_2)} \\
   \end{array}
```

… Can we do better?

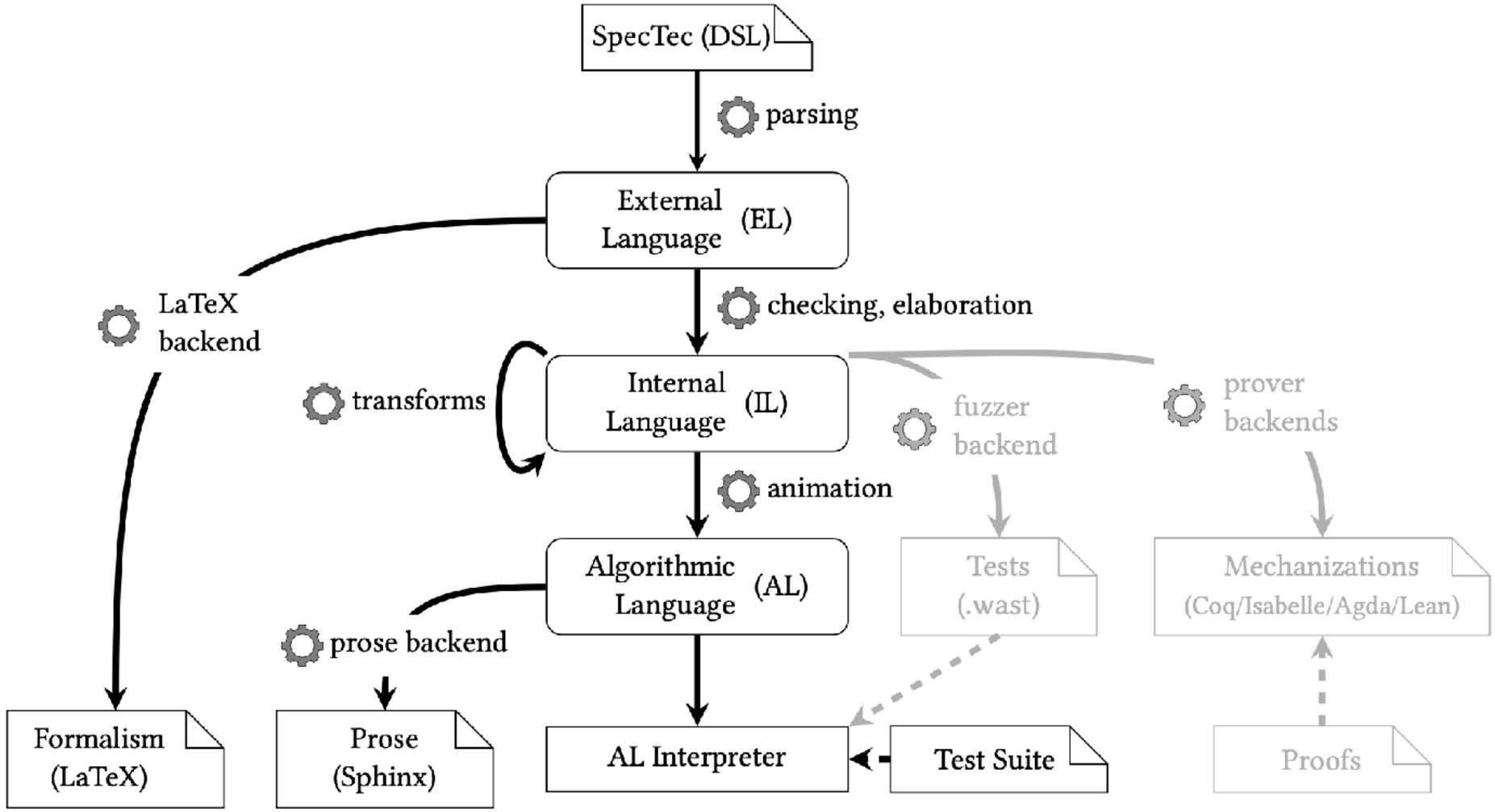
SpecTec

Framework for mechanizing the specification of Wasm
- Embracing both declarative and algorithmic styles of the semantics

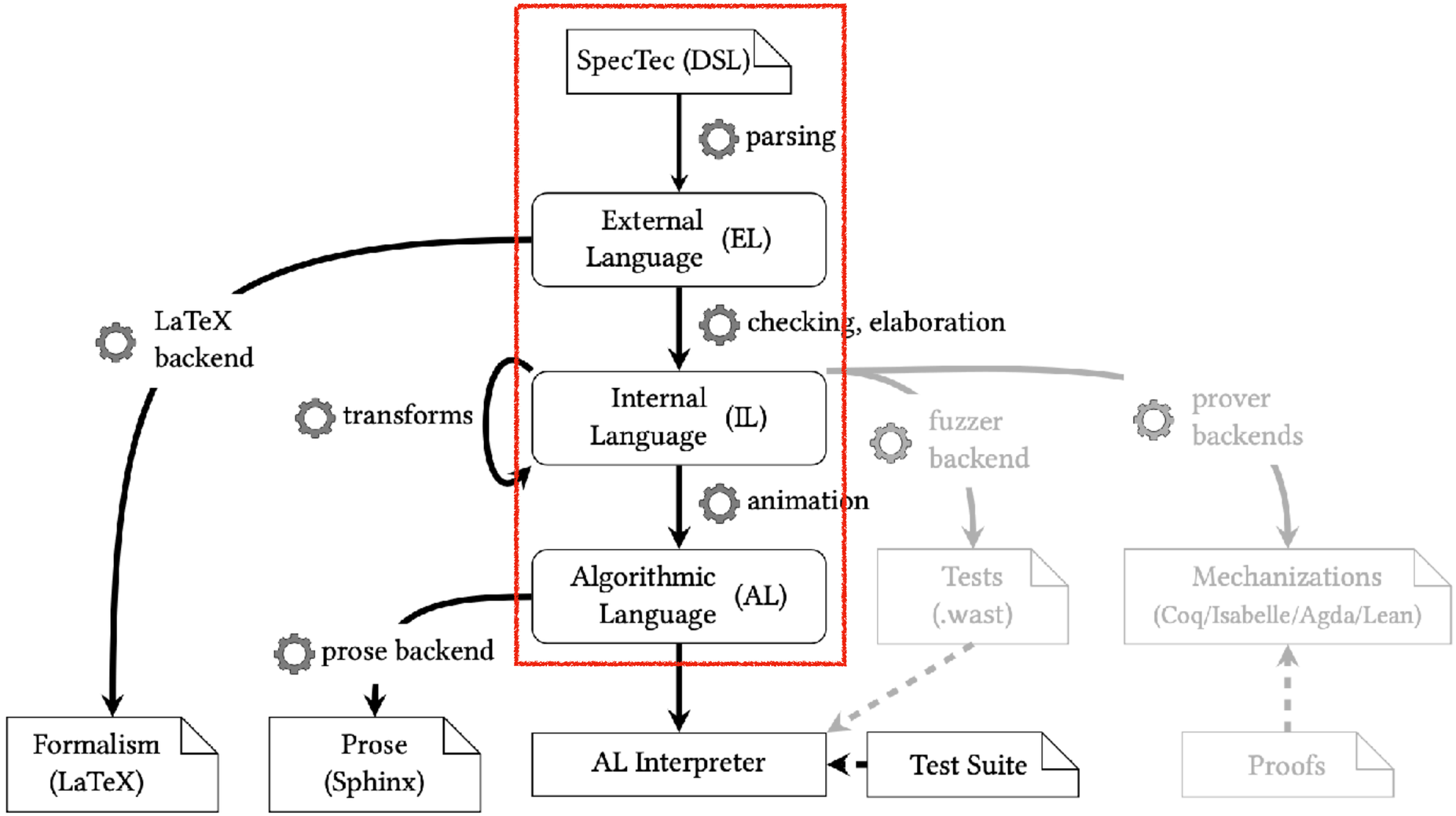
Provides a domain-specific language (DSL)
- Syntax, type system, and execution semantics of Wasm can be easily defined

The single source of truth from which various artifacts can be auto-generated
- LaTeX backend, Prose backend, Interpreter backend, …

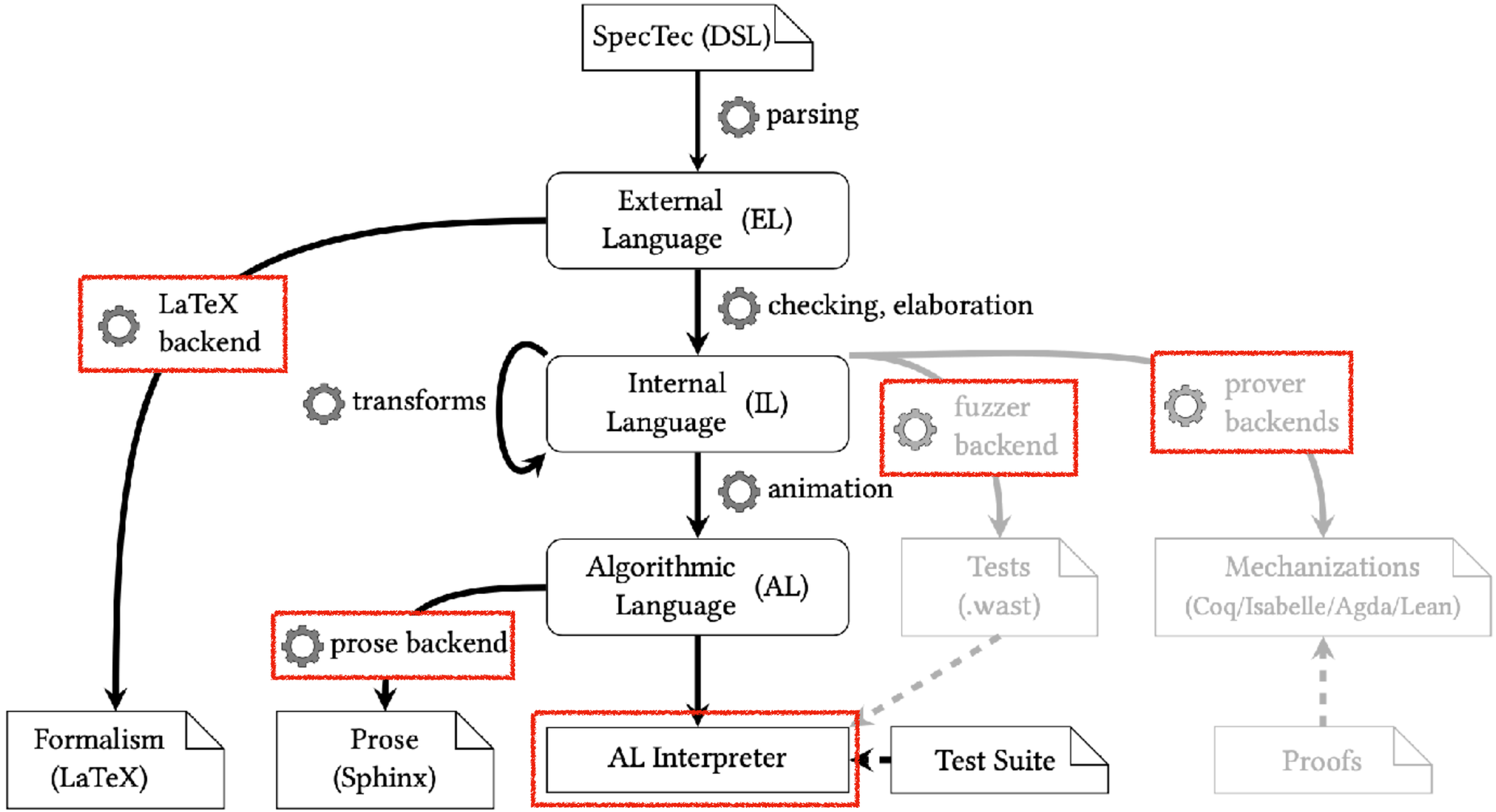
Overview


SpecTec (DSL)
parsing
External Language (EL)
checking, elaboration
LaTeX backend
transforms
Internal Language (IL)
fuzzer backend
prover backends
animation
Algorithmic Language (AL)
Tests (.wast)
Mechanizations (Coq/Isabelle/Agda/Lean)
prose backend
Formalism (LaTeX)
Prose (Sphinx)
AL Interpreter
Test Suite
Proofs

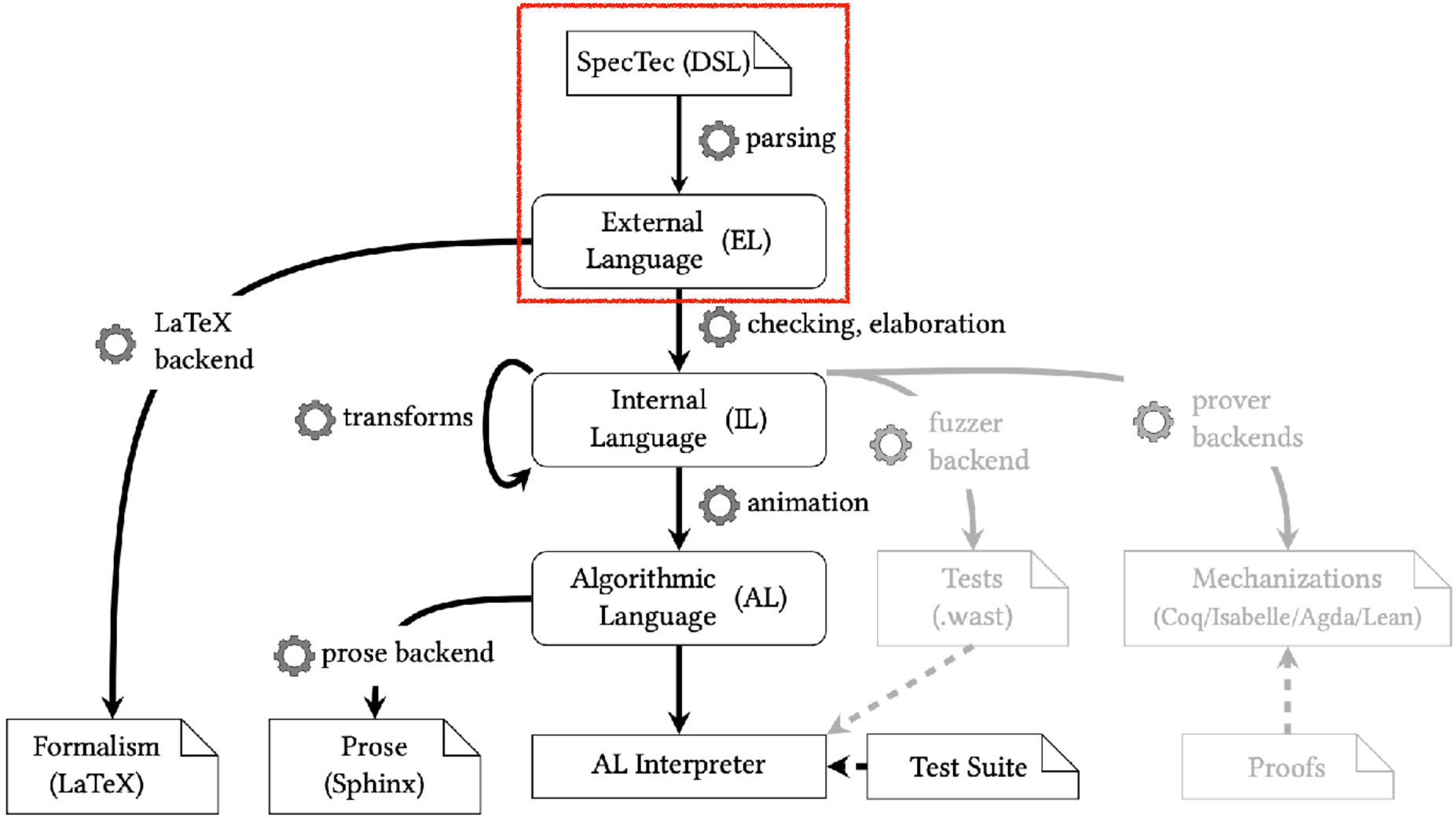
Overview: Backbone



Overview: Backends



DSL → EL


SpecTec (DSL)
parsing
External Language (EL)
checking, elaboration
LaTeX backend
transforms
Internal Language (IL)
fuzzer backend
prover backends
animation
Algorithmic Language (AL)
Tests (.wast)
Mechanizations (Coq/Isabelle/Agda/Lean)
prose backend
Formalism (LaTeX)
Prose (Sphinx)
AL Interpreter
Test Suite
Proofs

DSL

Designed to concisely describe the Wasm specification

Closely resembles pen-and-paper notation

```
syntax instr = …
  | CONST numtype num_(numtype)
  | UNOP numtype unop_(numtype)
  | BINOP numtype binop_(numtype)
  | TESTOP numtype testop_(numtype)
  | RELOP numtype relop_(numtype)
  | ...
```

<Sytax>

```
rule Instr_ok/relop:
  C |- RELOP nt relop_nt : nt nt -> I32
```

<Relation: Typing Rule>

```
def $relop(numtype, relop, num, num) : num_(I32)
def $relop(inn, EQ, iN_1, iN_2) =
  $ieq($size(inn), iN_1, iN_2)
def $relop(inn, NE, iN_1, iN_2) =
  $ine($size(inn), iN_1, iN_2)
```

<Function>

```
rule Step_pure/relop:
  (CONST nt c_1) (CONST nt c_2) (RELOP nt relop)
~> (CONST I32 c)
  -- if c = $relop(nt, relop, c_1, c_2)
```

<Relation: Execution Semantics>

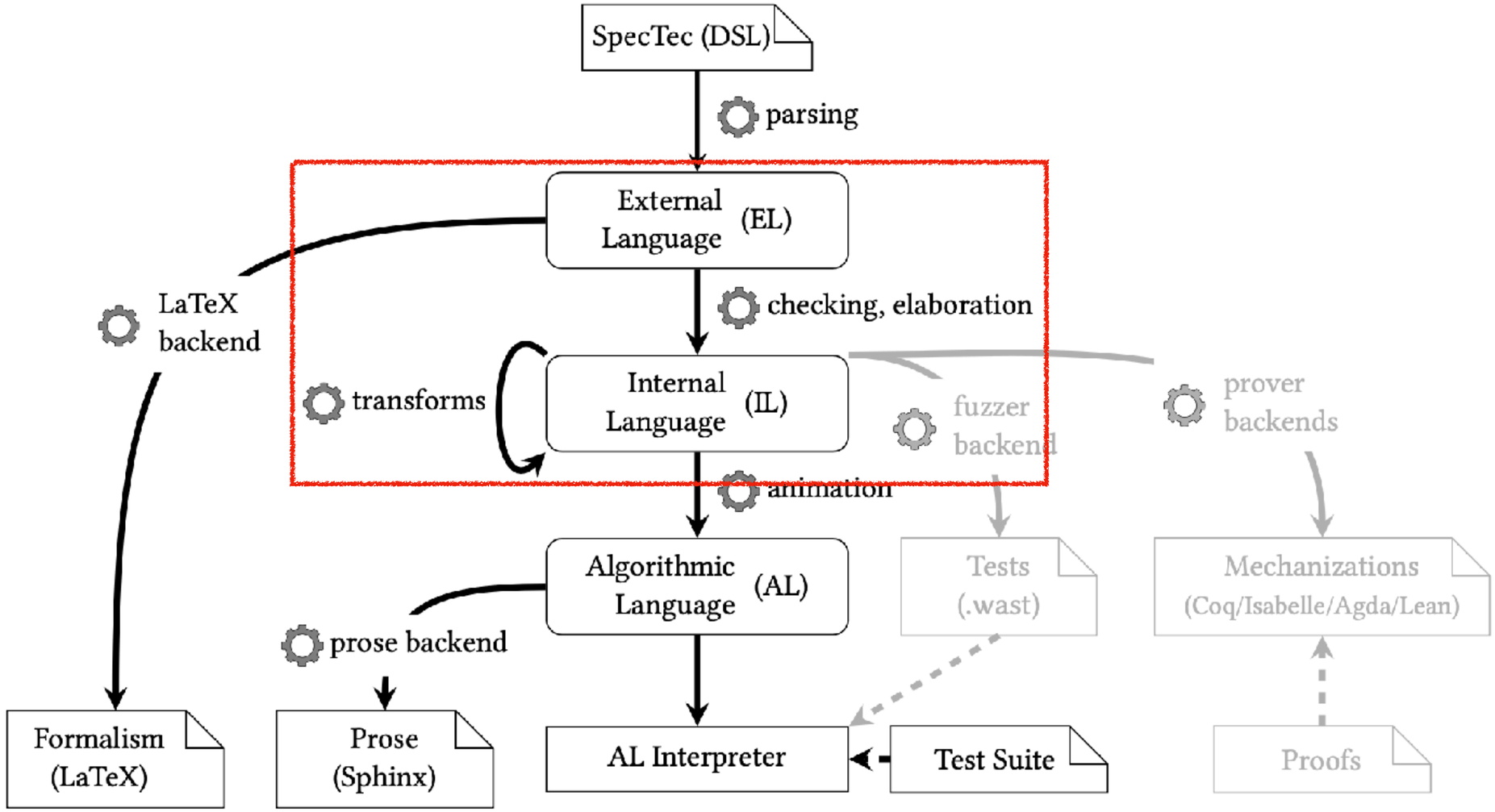
DSL → EL

Definitions in DSL are parsed into a representation called External Language (EL)

Identifier	*id*	::=	…lower-case-identifier …
Atom	*atom*	::=	…upper-case-identifier … | –> | ~> | : | |– | …
Operator	*unop*	::=	~ | + | –
	binop	::=	/\ | \/ | <=> | + | – | * | / | ^ | = | =/= | < | <= | >= | > | …
Iteration	*iter*	::=	? | * | + | ^*exp*
Type	*typ*	::=	*id* | **bool** | **nat** | *typ typ* | *typ iter* | *atom* | { (*atom typ*,)* } | (| *atom typ*)+ | (*typ*,*) | …
Expression	*exp*	::=	*id* | *bool* | *num* | *unop exp* | *exp binop exp* | $*id exp*? | **eps** | *exp exp* | *exp iter* | *exp*[*exp*]
		|	|*exp*| | *atom* | { (*atom exp*,)* } | *exp.atom* | *exp*[*path* = *exp*] | (*exp*,*) | …
Path	*path*	::=	*path*?[*exp*] | *path*?.*atom* | …
Definition	*def*	::=	**syntax** *id* = *typ* | **relation** *id* : *typ* | **rule** *id* (/ *id*)* : *exp* (-- *prem*)*
		|	**def** $*id exp*? : *typ* | **def** $*id exp*? = *exp* (-- *prem*)* | **var** *id* : *typ*
Premise	*prem*	::=	*id* : *exp* | **if** *exp* | **otherwise** | *prem iter*

<Syntax of EL>

EL → IL


SpecTec (DSL)
parsing
External Language (EL)
checking, elaboration
LaTeX backend
transforms
Internal Language (IL)
fuzzer backend
prover backends
animation
Algorithmic Language (AL)
Tests (.wast)
Mechanizations (Coq/Isabelle/Agda/Lean)
prose backend
Formalism (LaTeX)
Prose (Sphinx)
AL Interpreter
Test Suite
Proofs

EL → IL

EL is elaborated into a more explicit and unambiguous Internal Language (IL)

* Type checking
* Dimension inference
* Binding and recursion analysis
* Type-driven resolution of notational overloading

✘

```
-- if x* = [1, 2, 3]
-- if y* = [0, 1, 2]
-- if x* < y* + 1
```

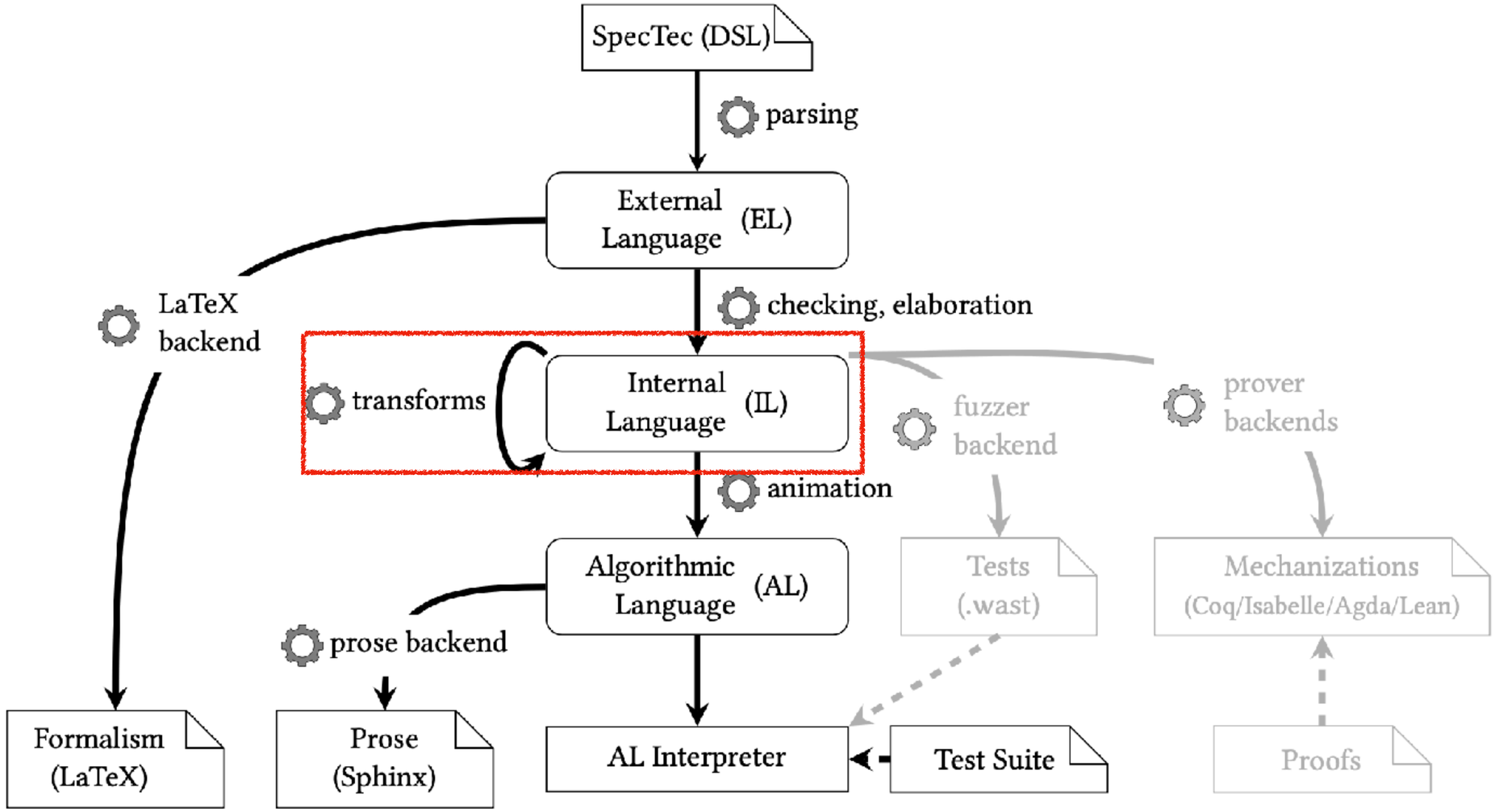
Type checking fail

✔

```
-- if x* = [1, 2, 3]
-- if y* = [0, 1, 2]
-- if (x < y + 1)*
```

Type checking + Dimension inference

IL → IL

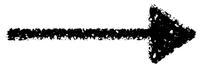

SpecTec (DSL)
parsing
External Language (EL)
checking, elaboration
LaTeX backend
transforms
Internal Language (IL)
fuzzer backend
prover backends
animation
Algorithmic Language (AL)
Tests (.wast)
Mechanizations (Coq/Isabelle/Agda/Lean)
prose backend
Formalism (LaTeX)
Prose (Sphinx)
AL Interpreter
Test Suite
Proofs

IL → IL

Runs a number of transformations on the IL for simplification / explicit notations

* Infer implicit side conditions
* Lift partial function types into options
* Introduce auxiliary variables for option types
* Turn subsumption into explicit injections
* Identifying variable bindings + reorder premises

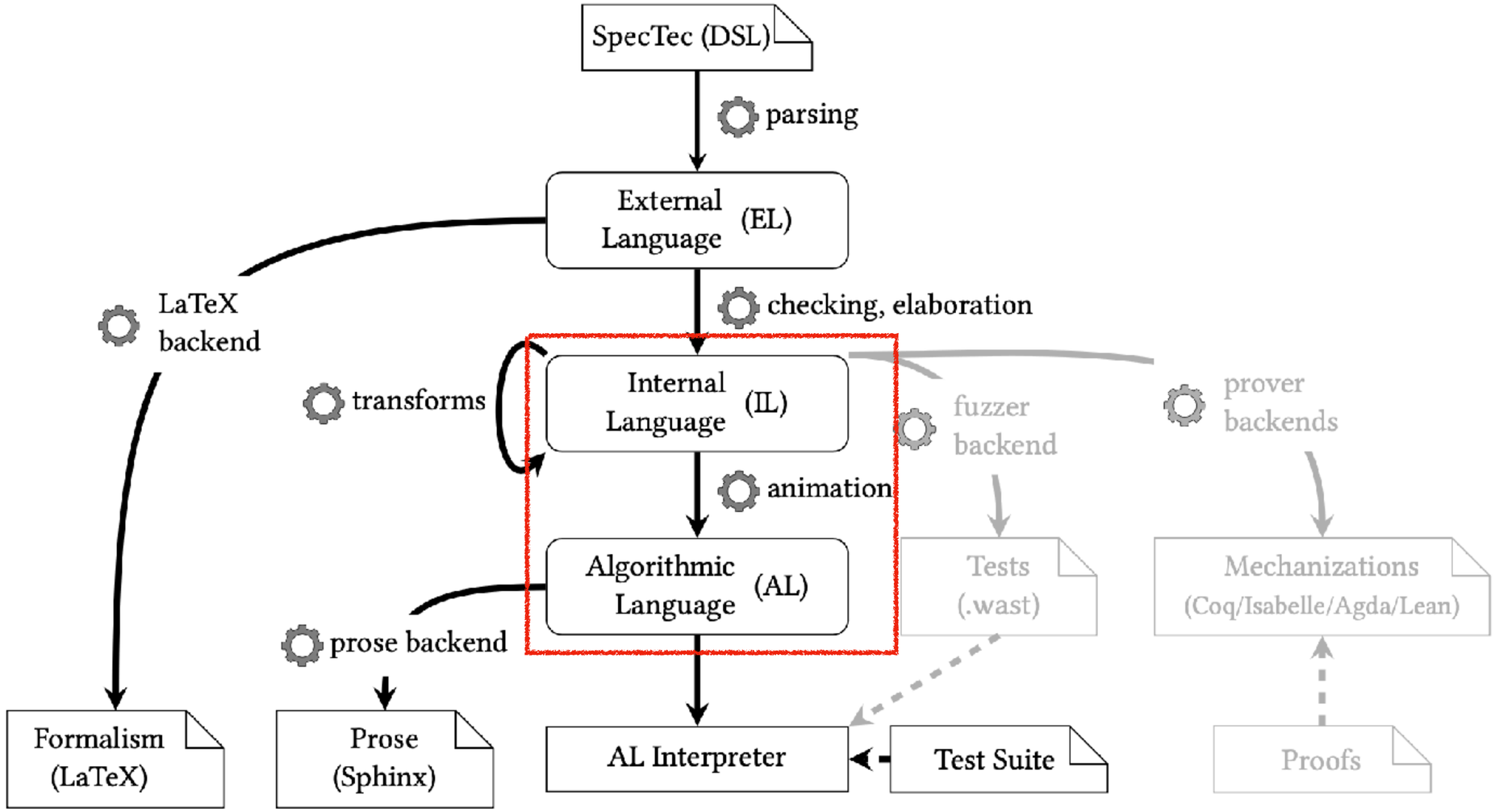
```
-- if x* = [1, 2, 3]
-- if y* = [0, 1, 2]
-- if (x < y + 1)*
```



```
-- if x* = [1, 2, 3]
-- if y* = [0, 1, 2]
-- if (x < y + 1)*
-- if |x*| = |y*|
```

Inferred side condition

IL → AL


SpecTec (DSL)
parsing
External Language (EL)
checking, elaboration
LaTeX backend
transforms
Internal Language (IL)
fuzzer backend
prover backends
animation
Algorithmic Language (AL)
Tests (.wast)
Mechanizations (Coq/Isabelle/Agda/Lean)
prose backend
Formalism (LaTeX)
Prose (Sphinx)
AL Interpreter
Test Suite
Proofs

AL

Algorithmic Language (AL) represents runtime semantics in an imperative style, from which prose/interpreter can be easily generated

Program	p	::=	a^*
Algorithm	a	::=	algorithm $x(e^*)$ s^*
Statement	s	::=	if c s^* s^* — If c, then: s_1^* Else: s_2^*
		|	enter e e s^* — Enter e_1 with e_2 : s^*.
		|	exit — Exit current context.
		|	assert c — Assert: c.
		|	push e — Push e to the stack.
		|	pop e — Pop e from the stack.
		|	let e e — Let e_1 be e_2.
		|	trap — Trap.
		|	nop — Do nothing.
		|	return $e^?$ — Return $e^?$.
		|	execute e — Execute e.
		|	perform x e^* — Perform $x(e^*)$.
		|	replace e q^* e — Replace $e_1[q^*]$ with e_2.

Condition	c	::=	not c | c $\oplus$ c | iscase e x | w	
Expression	e	::=	x	x
		|	n	n
		|	$\oplus$ e	$\oplus$ e
		|	e $\oplus$ e	e_1 $\oplus$ e_2
		|	$e[q]$	$e[q]$
		|	$e[q^* := e]$	e_1 with q^* = e_2
		|	$\{(x \mapsto e)^*\}$	{ $(x$: $e)^*$ }
		|	$[e^*]$	e^*
		|	e ++ e	e_1 e_2
		|	$\|e\|$	length of e
		|	$(k$ $e^*)$	$(k$ $e^*)$
		|	$x(e^*)$	$x(e^*)$
		|	w	
Path	q	::=	e | $.x$	

<Syntax of AL>

IL → AL

Translating IL definitions into AL definitions is mostly straightforward,

after some of preprocessing is performed

```
LHS   ~>   RHS
-- PREM*
```

<Preprocessed IL>

```
1. Pop … from the stack.
2. Let … be ….
3. If …, then:
   a. Let … be ….
   b. Push … to the stack.
   c. Execute ….
```

<AL>

[Preprocess] 1. Anti-unification of LHS

If two reduction rules for one instruction have different LHS,
they should be anti-unified

```
… (br 0)   ~>   RHS1
-- PREM1*
```

```
… (br n+1)   ~>   RHS2
-- PREM2*
```

```
… (br m)   ~>   RHS1
-- if m = 0
-- PREM1*
```

```
… (br m)   ~>   RHS2
-- if m > 0
-- if n = m - 1
-- PREM2*
```

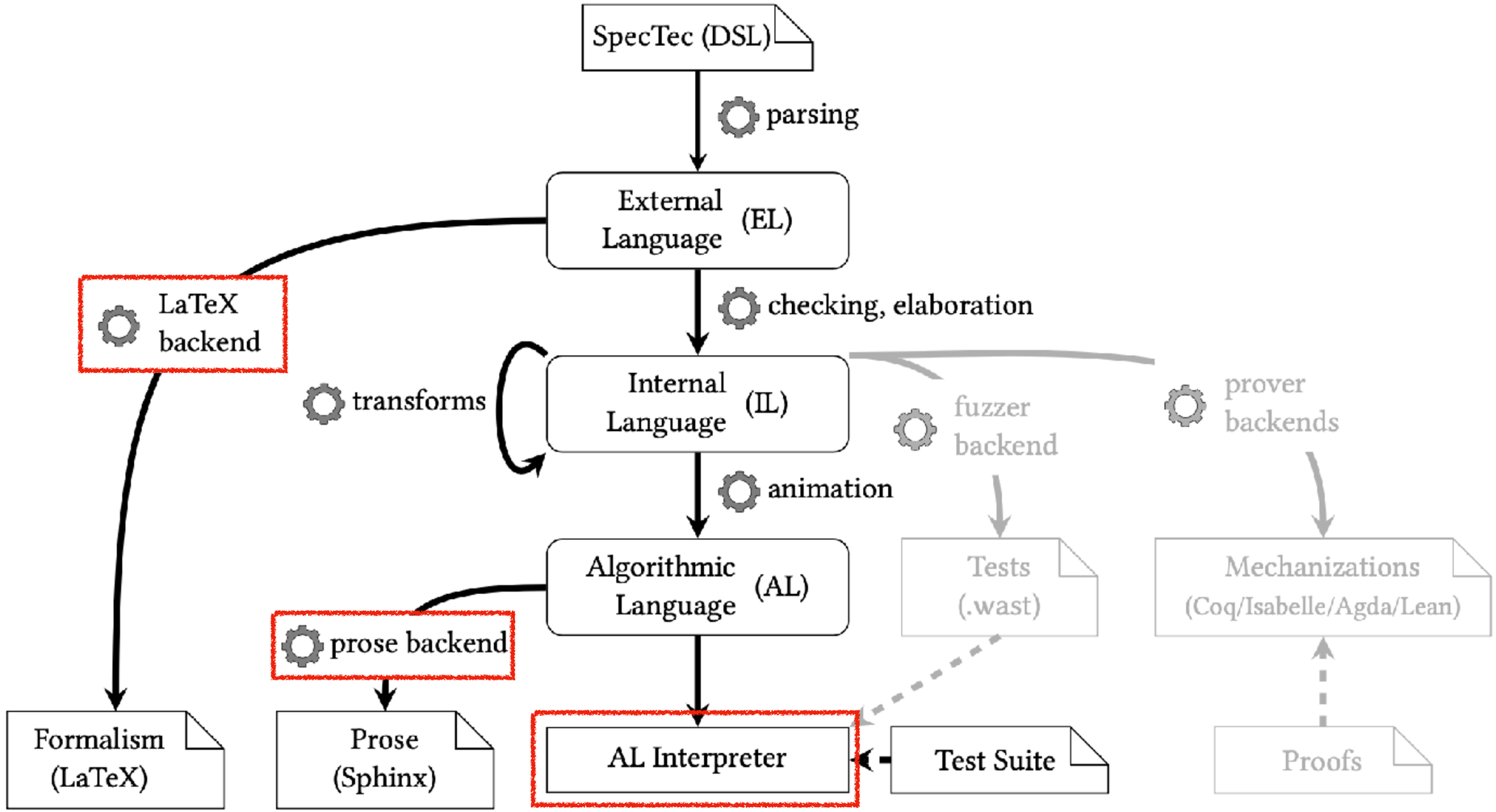
[Preprocess] 2. Identifying Binding Occurrences

Goal: Identify whether each equality premise is
a binding occurrence or an equality-checking,
so that each variable is bound exactly once (if possible)

```
… x …  ~>  RHS
-- if (a, b) = x
-- if a = b
```

```
… x …  ~>  RHS
-- where (a, b) = x
-- if a == b
```

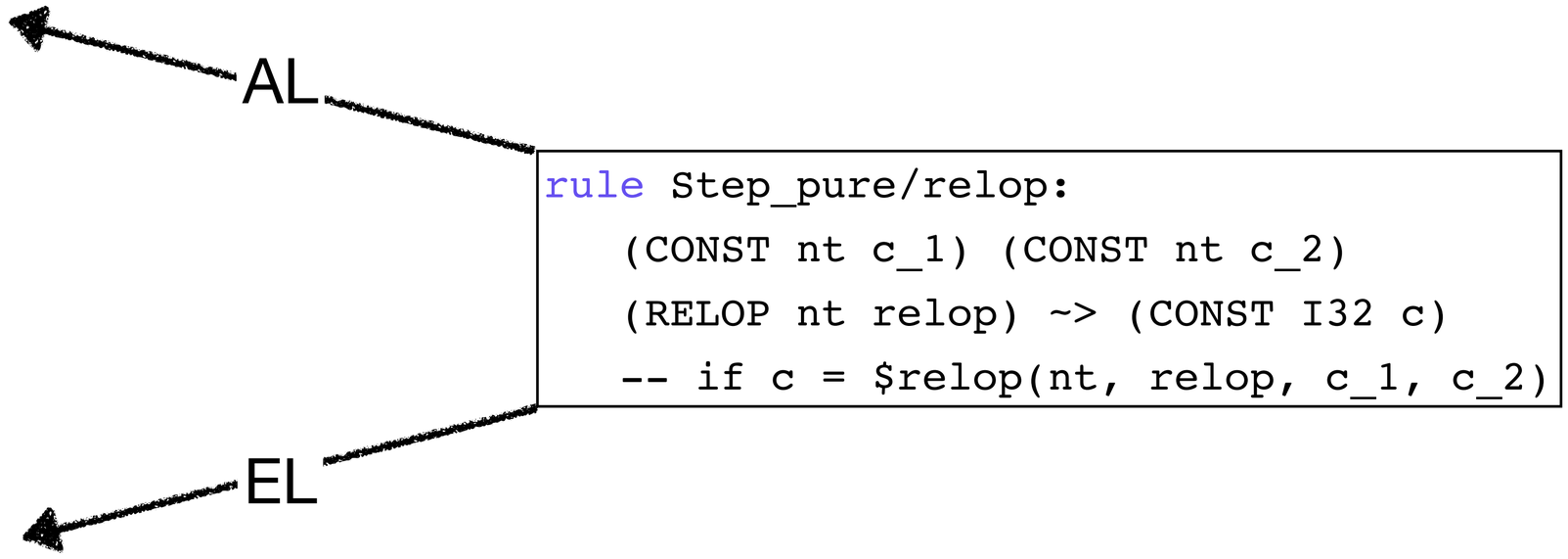
Backends


SpecTec (DSL)
parsing
External Language (EL)
checking, elaboration
LaTeX backend
transforms
Internal Language (IL)
fuzzer backend
prover backends
animation
Algorithmic Language (AL)
Tests (.wast)
Mechanizations (Coq/Isabelle/Agda/Lean)
prose backend
Formalism (LaTeX)
Prose (Sphinx)
AL Interpreter
Test Suite
Proofs

Formal / Prose Backends

Input:

```
.. _exec-relop:

$${rule-prose: exec/relop}

\

$${rule: Step_pure/relop}
```



Formal / Prose Backends

Output:

```
.. _exec-relop:

:math:`{\mathit{nt}} . {\mathit{relop}}`
.....................................
1. Assert: Due to validation, a value of value type :math:`{\mathit{nt}}` is on the top of the stack.
2. Pop the value :math:`({\mathit{nt}}{.}\mathsf{const}~c_2)` from the stack.
3. Assert: Due to validation, a value of value type :math:`{\mathit{nt}}` is on the top of the stack.
4. Pop the value :math:`({\mathit{nt}}{.}\mathsf{const}~c_1)` from the stack.
5. Let :math:`c` be :math:`{{{\mathit{relop}}}{}_{{\mathit{nt}}}}{(c_1,\, c_2)}`.
6. Push the value :math:`(\mathsf{i{\scriptstyle32}}{.}\mathsf{const}~c)` to the stack.

\

.. math::
   \begin{array}{@{}l@{}rcl@{}l@{}}
   & ({\mathit{nt}}{.}\mathsf{const}~c_1)~({\mathit{nt}}{.}\mathsf{const}~c_2)~({\mathit{nt}} .
{\mathit{relop}}) &\hookrightarrow& (\mathsf{i{\scriptstyle32}}{.}\mathsf{const}~c)
     &\qquad \mbox{if}~c = {{{\mathit{relop}}}{}_{{\mathit{nt}}}}{(c_1,\, c_2)} \\
   \end{array}
```

Formal / Prose Backends

Rendered:

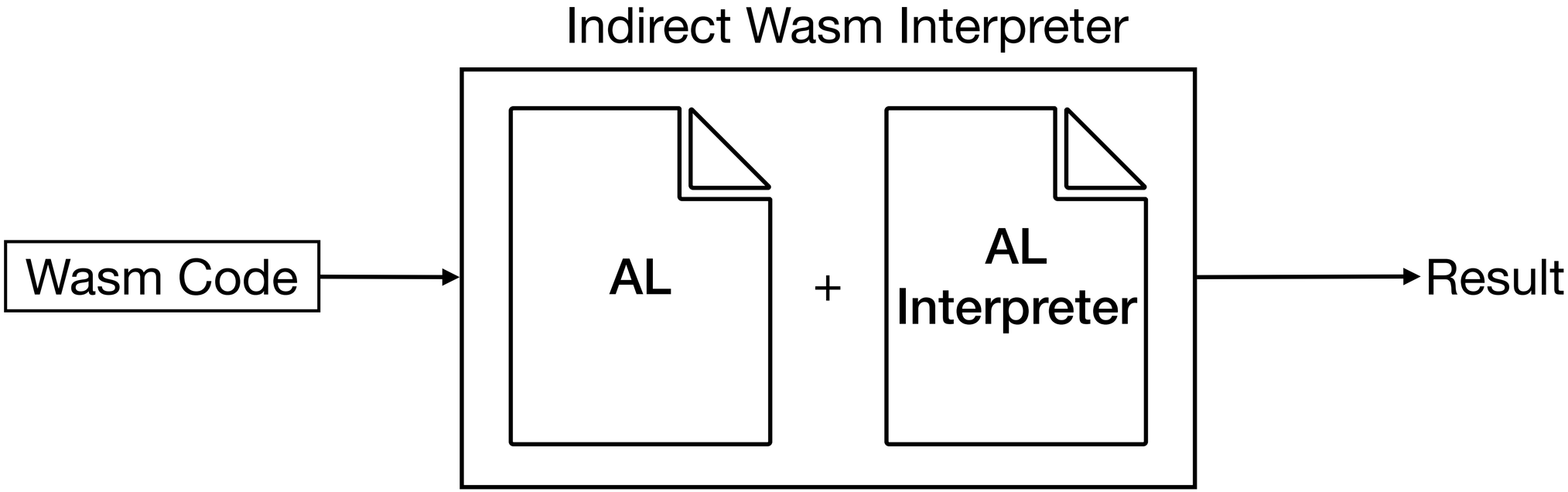
nt.relop

1. Assert: Due to validation, a value of value type nt is on the top of the stack.
2. Pop the value $(nt.\mathsf{const}\ c_2)$ from the stack.
3. Assert: Due to validation, a value of value type nt is on the top of the stack.
4. Pop the value $(nt.\mathsf{const}\ c_1)$ from the stack.
5. Let c be $relop_{nt}(c_1, c_2)$.
6. Push the value $(\mathsf{i32}.\mathsf{const}\ c)$ to the stack.

$$(nt.\mathsf{const}\ c_1)\ (nt.\mathsf{const}\ c_2)\ (nt.relop) \quad \hookrightarrow \quad (\mathsf{i32}.\mathsf{const}\ c) \qquad \text{if } c = relop_{nt}(c_1, c_2)$$

Indirect Interpreter

Interpreting Wasm by interpreting the specification written in AL



Evaluation

1. **Correctness**

   Does SpecTec correctly generate formal and prose specifications,
   as well as the interpreter backend?

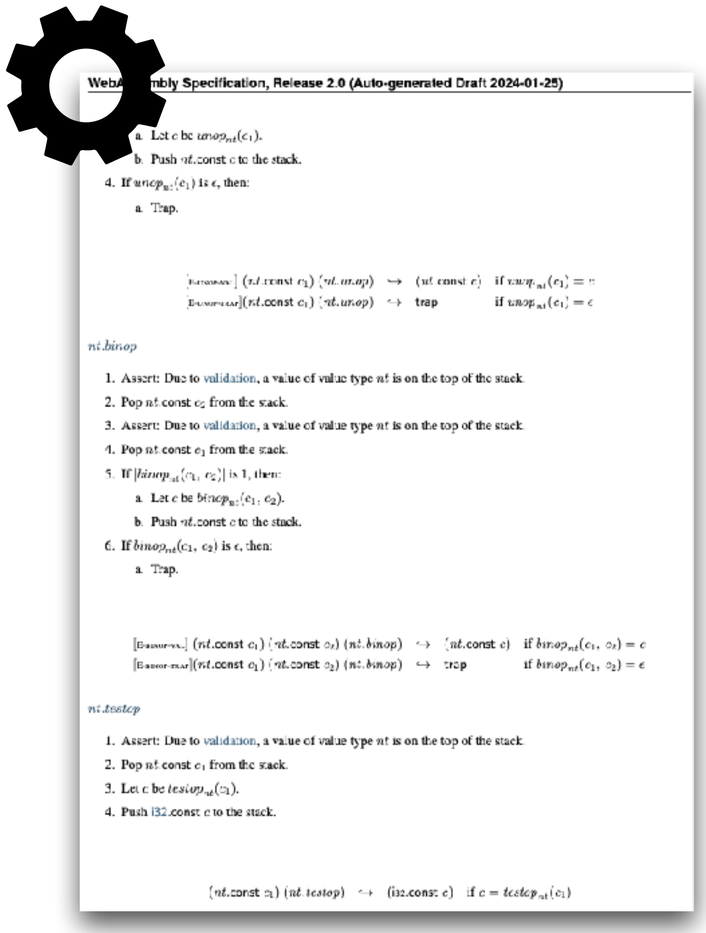
2. **Bug Prevention**

   Can SpecTec prevent bugs during Wasm standard development?

3. **Forward Compatibility**

   Can SpecTec support future language features?

1. Correctness



WebAssembly Specification, Release 2.0 (Auto-generated Draft 2024-01-25)

a. Let c be $unop_{nt}(c_1)$.

b. Push nt.const c to the stack.

4. If $unop_{nt}(c_1)$ is ϵ, then:

a. Trap.

$$\begin{array}{lrll} [\text{Binop-val}] & (nt.\text{const } c_1)\ (nt.unop) & \hookrightarrow & (nt.\text{const } c) \quad \text{if } unop_{nt}(c_1) = c \\ [\text{Binop-trap}] & (nt.\text{const } c_1)\ (nt.unop) & \hookrightarrow & \text{trap} \quad\quad\ \ \text{if } unop_{nt}(c_1) = \epsilon \end{array}$$

nt.binop

1. Assert: Due to validation, a value of value type nt is on the top of the stack.
2. Pop nt.const c_2 from the stack.
3. Assert: Due to validation, a value of value type nt is on the top of the stack.
4. Pop nt.const c_1 from the stack.
5. If $|binop_{nt}(c_1, c_2)|$ is 1, then:
   a. Let c be $binop_{nt}(c_1, c_2)$.
   b. Push nt.const c to the stack.
6. If $binop_{nt}(c_1, c_2)$ is ϵ, then:
   a. Trap.

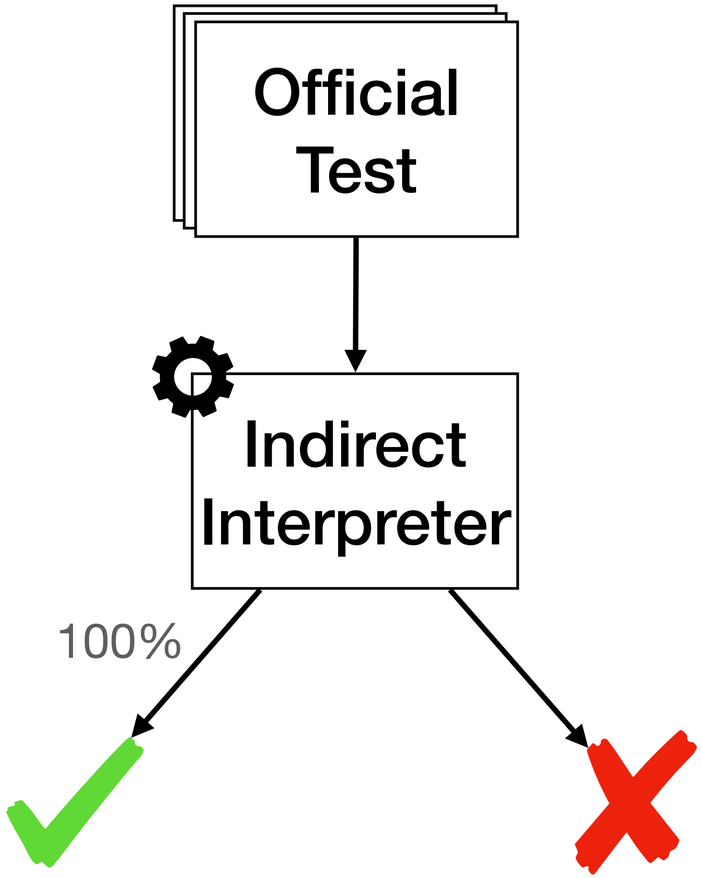
$$\begin{array}{lrll} [\text{Binop-val}] & (nt.\text{const } c_1)\ (nt.\text{const } c_2)\ (nt.binop) & \hookrightarrow & (nt.\text{const } c) \quad \text{if } binop_{nt}(c_1, c_2) = c \\ [\text{Binop-trap}] & (nt.\text{const } c_1)\ (nt.\text{const } c_2)\ (nt.binop) & \hookrightarrow & \text{trap} \quad\quad\ \ \text{if } binop_{nt}(c_1, c_2) = \epsilon \end{array}$$

nt.testop

1. Assert: Due to validation, a value of value type nt is on the top of the stack.
2. Pop nt.const c_1 from the stack.
3. Let c be $testop_{nt}(c_1)$.
4. Push i32.const c to the stack.

$$(nt.\text{const } c_1)\ (nt.testop) \quad \hookrightarrow \quad (\text{i32.const } c) \quad \text{if } c = testop_{nt}(c_1)$$

Auto-generation of formal / prose specification for Wasm 2.0



Auto-generation of an indirect interpreter + 100% pass rate of the official test

2. Bug Prevention

Injected 13 retrospective spec bugs into the spec written in DSL

→ All bugs were detected at various phases of SpecTec

[spec] Fix variable name typos (#1358)

[spec] Fix missing immediate on table.set (#1441)

[spec] Fix typos in instruction validation rules (#1462)

[spec] Fix naming typo (#1532)

[spec] Fix typo in element execution (#1544)

[spec] Add missing case for declarative elem segments

[spec] Remove an obsolete exec step (#1580)

[spec] Remove stray `x` indices (#1598)

[spec] Add missing value to table.grow reduction rule (#1607)

[spec] Fix reduction rule for label (#1612)

[spec] Add missing access to current frame in prose (#1624)

[spec] Add missing type to elem.drop and store soundness (#1668)

[spec] Pop dummy frame after Invocation (#1691)

3. Forward Compatibility

SpecTec can handle five proposals to be included in Wasm 3.0

+ Found & reported 10 bugs in the proposals

Phase 4 - Standardize the Feature (WG)

Proposal	Champion
Tail call	Andreas Rossberg
Extended Constant Expressions	Sam Clegg
Typed Function References	Andreas Rossberg
Garbage collection	Andreas Rossberg
Multiple memories	Andreas Rossberg

Pass index argument to `getfield` function #463

Fix `array.get/set` reduction rule #464

[spec] Fix spec for execution of `struct.new`, `array.new_fixed` and `br_on_cast(_fail)` #456

Dimension mismatch in the premise of `array.new_data` reduction rule #476

Minor changes on array.new_elem #477

Fix ref.null semantics #478

Add current frame #484

Wasm Specifications and Proposals Have Bugs!

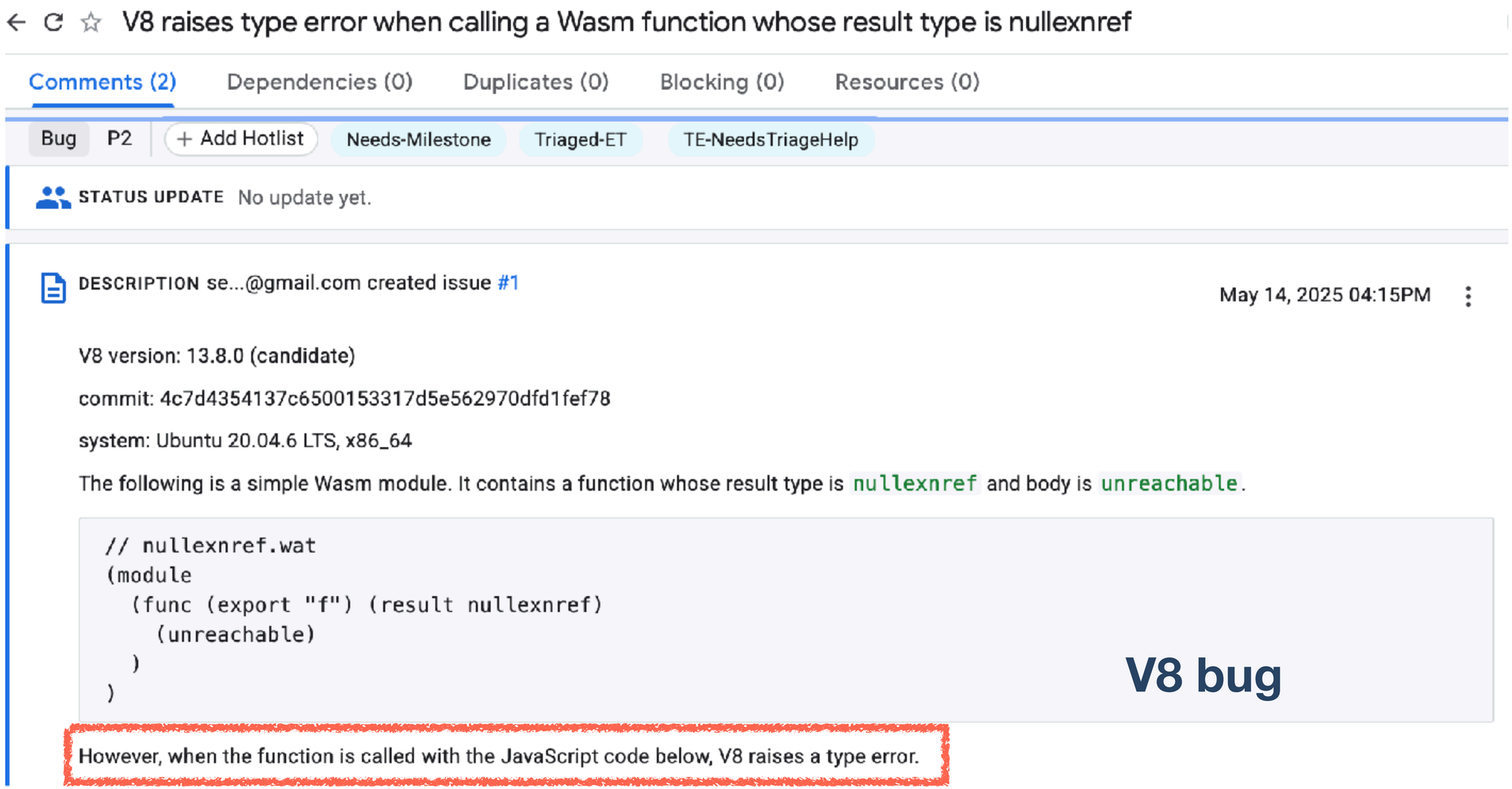
- Type errors
  - Missing immediates or record fields in formal rules
- Semantic errors
  - Missing stack operands, stack mishandling, index errors in formal rules
- Prose errors
  - Unbound variables, missing steps, other mistranslation
- Editorial errors
  - Typos, layout errors, wrong hyperlinks, …

Interactions of Individually Tested Proposals


```
(module
  (type $arr_v128 (array v128))
  (func $f (export "f") (result v128)
    (ref.null $arr_v128)
    (i32.const 0)
    (array.get $arr_v128))
)
(assert_trap (invoke "f") "")
```

JavaScriptCore bug

Interoperations with Different Languages



V8 raises type error when calling a Wasm function whose result type is nullexnref

Comments (2) Dependencies (0) Duplicates (0) Blocking (0) Resources (0)

Bug P2 + Add Hotlist Needs-Milestone Triaged-ET TE-NeedsTriageHelp

STATUS UPDATE No update yet.

DESCRIPTION se...@gmail.com created issue #1 May 14, 2025 04:15PM

V8 version: 13.8.0 (candidate)

commit: 4c7d4354137c6500153317d5e562970dfd1fef78

system: Ubuntu 20.04.6 LTS, x86_64

The following is a simple Wasm module. It contains a function whose result type is `nullexnref` and body is `unreachable`.

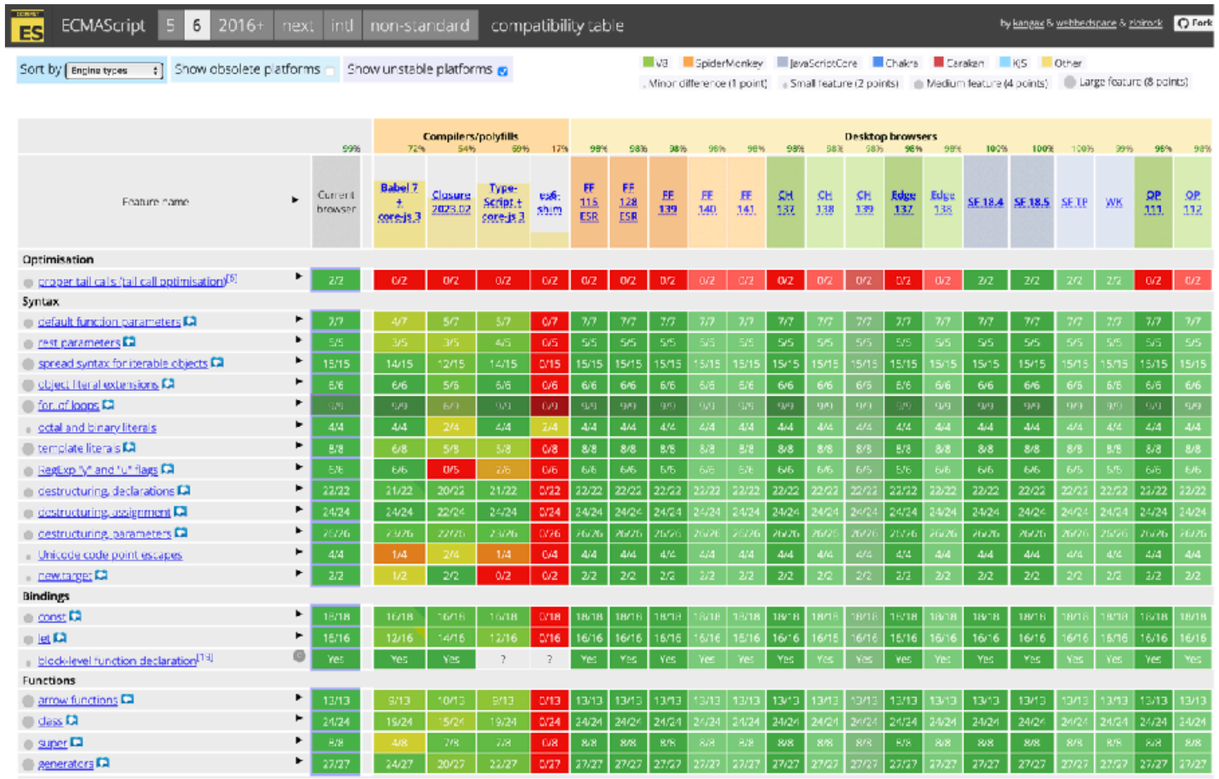
```
// nullexnref.wat
(module
  (func (export "f") (result nullexnref)
    (unreachable)
  )
)
```

V8 bug

However, when the function is called with the JavaScript code below, V8 raises a type error.

https://issues.chromium.org/issues/417604765

Different Feature Support by Implementations



https://compat-table.github.io/compat-table/es6/

	Your browser	Chrome	Firefox	Safari	Node.js	Deno	GraalWasm	Chicory	Wasmtime	Wasmer	wasm2c
Phase 5 - The Feature is Standardized											
JS BigInt to Wasm i64 Integration	✓	85	78	15[h]	15.0	1.1.2	21.3	N/A	N/A	N/A	N/A
Branch Hinting	?	137	[d]	16	[i]	[i]	×	×	×	×	×
Bulk Memory Operations	✓	75	79	15	12.5	0.4	23.0	1.0.0	0.20	1.0	1.0.30
Custom Text Format Annotations	?	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✓	✓	N/A
Extended Constant Expressions	✓	114	112	17.4	21.0	1.33	[an]	N/A	25	×	[am]
Garbage Collection	✓	119	120	18.2	22.0	1.38	×	×	[ag]	×	×
Multiple Memories	×	120	125	×	22.0	1.38	[ac]	N/A	15	×	[ao]
Multi-value	✓	85	78	13.1	15.0	1.3.2	22.3	1.0.0	0.17	1.0	1.0.24
Import/Export of Mutable Globals	✓	74	61	12	12.0	0.1	21.3	1.0.0	✓	0.7	1.0.1
Reference Types	✓	96	79	15	17.2	1.16	23.0	1.0.0	0.20	2.0	1.0.31

https://webassembly.org/features/

Agenda for the October meeting of WebAssembly's Community Group

- **Host:** Google, München, Germany
- **Dates:** Wednesday-Thursday, October 11-12, 2023
- **Times:** 9:00AM - 5:00PM

Initial work on a DSL for the specification document

SR presenting slides [TODO: SR to link slides]

TL: very cool, want to write less latex. Would be nice to have better spec-tests. Would it be possible to generate tests that don't just find bugs, but look pretty and organized well and could be checked in.

SR: 100% pass rate in the current spec test. Next step is to find what kind of coverage metrics are useful and better sense of correctness.

FM: some work on specs that are an intersection of wasm and JS, suggestions how to improve that process. JS types that are very simple which go to and fro. JS API is not Wasm or JS but between them.

SR: we like those problems, branch of research for those problems. Interaction between DOM and JS, or embedded. In beginning when working on JS, it was difficult to find bugs. JS devs disagreed on what were bugs. Interested in interaction between wasm and JS. Until march of this year, we did not know about wasm. We're now on top of wasm 2.0, working on SIMD now. Research on interaction, component-model, benchmark(?). During break will ask more questions.

https://github.com/WebAssembly/meetings/blob/main/main/2023/CG-10.md

WEBASSEMBLY

Agenda for the June meeting of WebAssembly's Community Group

- **Host**: Carnegie Mellon University, Pittsburgh, Pennsylvania, USA
- **Dates**: Wednesday-Thursday, June 5-6, 2024

Poll: Adopt SpecTec (once it's ready) as the toolchain for authoring the spec.

SF: 19 in room, 5 on chat F: 13 in room, 4 online N: 4 in room, 4 online A: 1 in room SA: 0

DS: Consensus.

WEBASSEMBLY

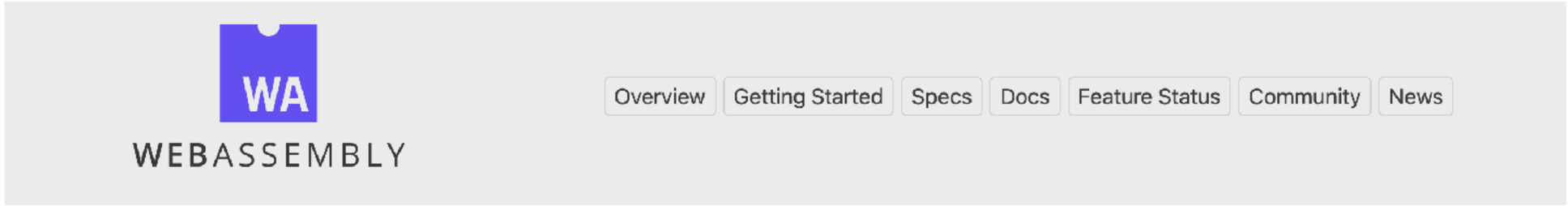
Agenda for the March 11 video call of WebAssembly's Community Group

- **Where**: Virtual meeting
- **When**: March 11, 16:00-17:00 UTC (9am-10am PDT, 17:00-18:00 CET)

Poll: Adopt spectec as the toolchain for authoring the wasm spec.

SF: 15 F: 17 N: 1 A: 0 SA: 0

TL: That is consensus.



SpecTec has been adopted

Published on March 27, 2025 by Andreas Rossberg.

Two weeks ago, the Wasm Community Group voted to adopt SpecTec for authoring future editions of the Wasm spec. In this post, I'll shed some light on what SpecTec is, what it helps with, and why it takes Wasm to a new level of rigor and assurance that is unprecedented when it comes to language standards.

Formal language semantics

One feature that sets Wasm apart from other mainstream programming technologies is that it comes with a complete formalization: its syntax (binary and text format), type system (validation), and operational semantics (execution) are given in self-contained, mathematically precise terms. This formal specification was not an afterthought, it is an integral part of Wasm's design process and has been a normative part of the official language standard from day one. And not just that: it enabled the *soundness* of the language — i.e., the fact that it has no undefined behavior and no runtime type errors can occur — to be machine-verified before the first version of the standard was published.

This was a huge leap forward, because the practical state of language specifications is basically stuck in the 1960s: most language standards, even new ones, are still defined by some basic grammar notation for their syntax (and sometimes not even that), while their semantics is given by a combination of pretty prose, hidden assumptions, and wishful thinking.

https://webassembly.org/news/2025-03-27-spectec/

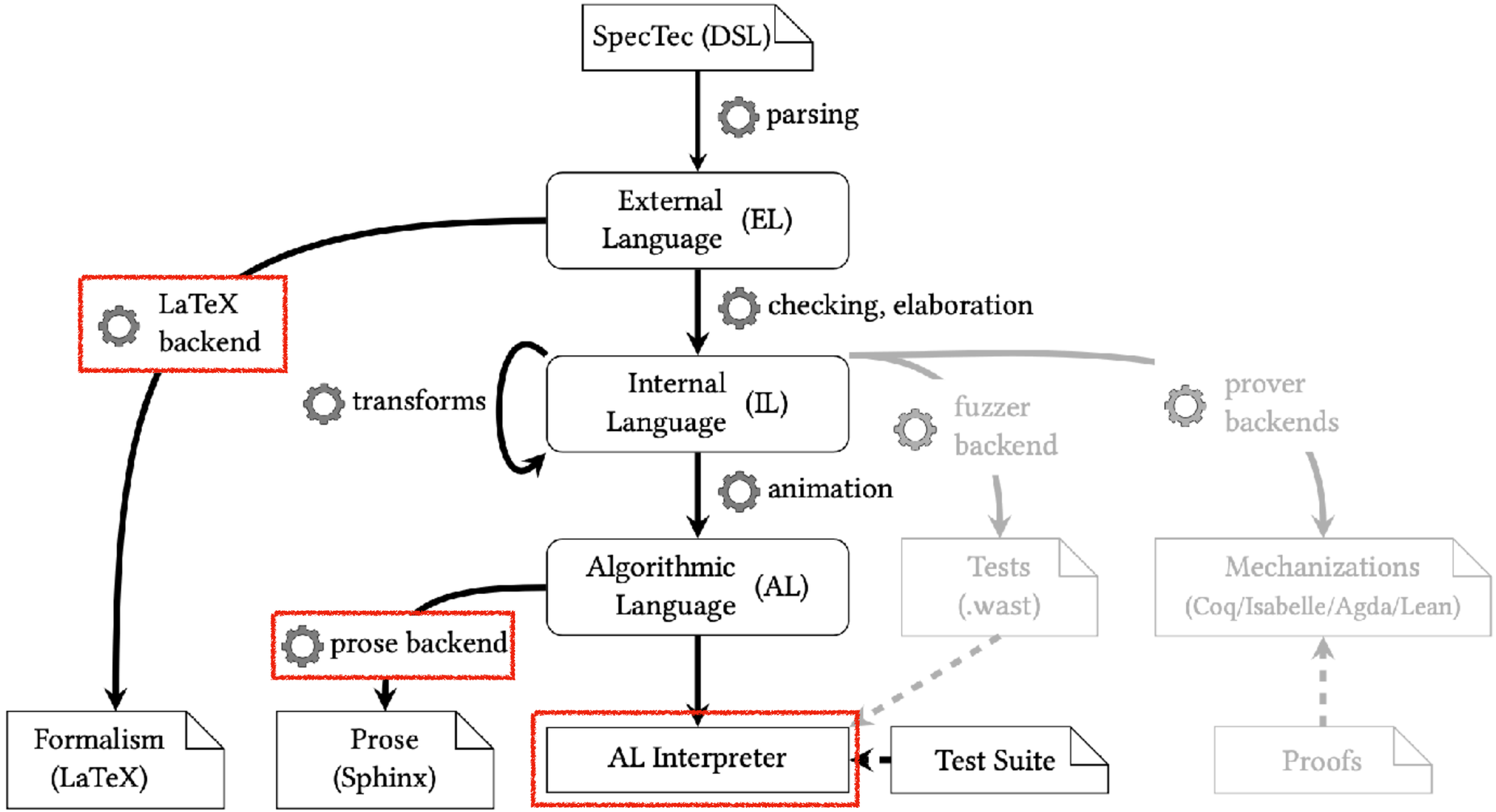
Wasm 3.0 Completed

Published on September 17, 2025 by Andreas Rossberg.

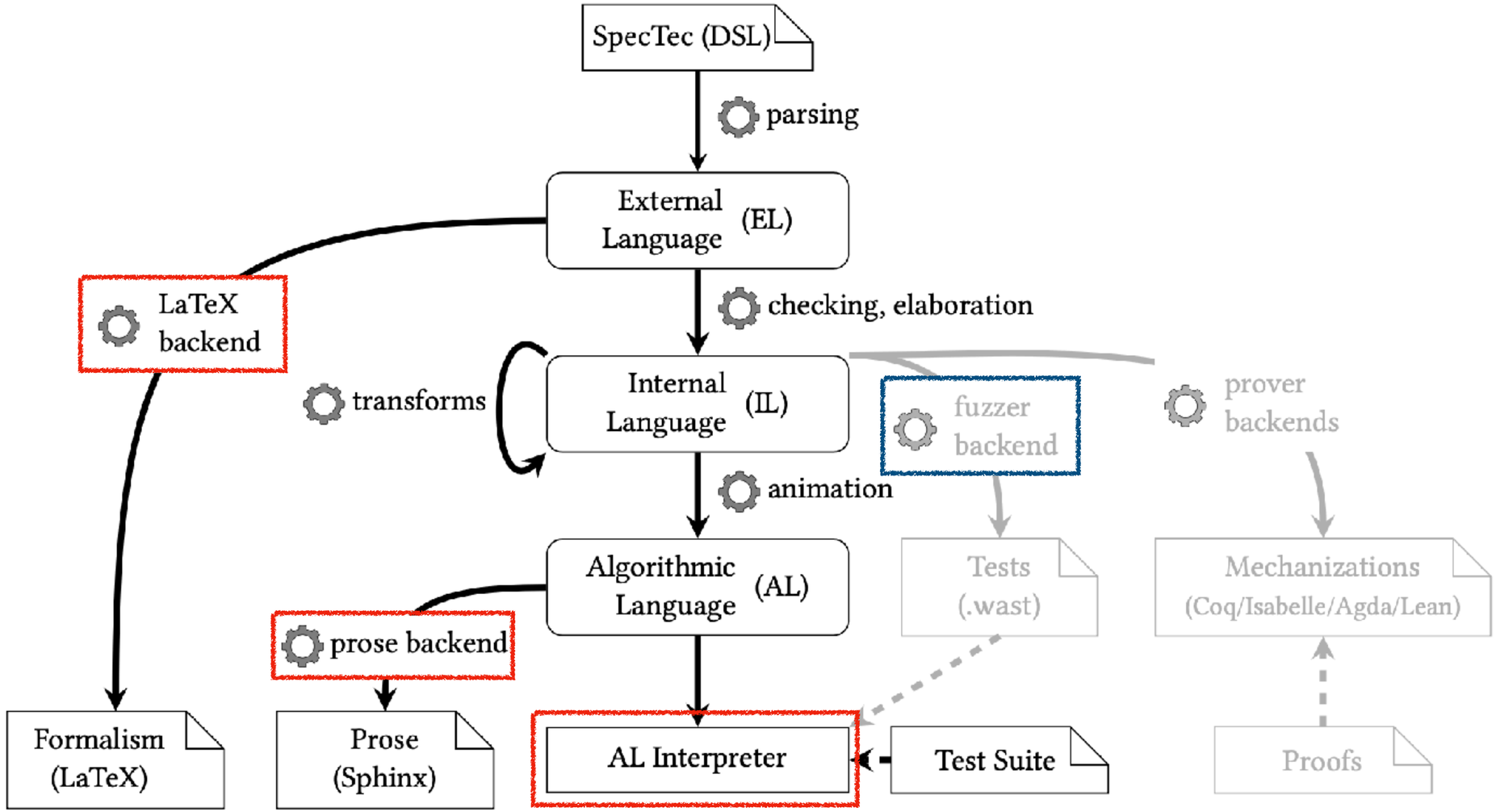
Three years ago, version 2.0 of the Wasm standard was (essentially) finished, which brought a number of new features, such as vector instructions, bulk memory operations, multiple return values, and simple reference types.

In the meantime, the Wasm W3C Community Group and Working Group have not been lazy. Today, we are happy to announce the release of Wasm 3.0 as the new "live" standard.

Backends


SpecTec (DSL)
parsing
External Language (EL)
checking, elaboration
LaTeX backend
transforms
Internal Language (IL)
fuzzer backend
prover backends
animation
Algorithmic Language (AL)
Tests (.wast)
Mechanizations (Coq/Isabelle/Agda/Lean)
prose backend
Formalism (LaTeX)
Prose (Sphinx)
AL Interpreter
Test Suite
Proofs

Backends


SpecTec (DSL)
parsing
External Language (EL)
checking, elaboration
LaTeX backend
transforms
Internal Language (IL)
fuzzer backend
prover backends
animation
Algorithmic Language (AL)
Tests (.wast)
Mechanizations (Coq/Isabelle/Agda/Lean)
prose backend
Formalism (LaTeX)
Prose (Sphinx)
AL Interpreter
Test Suite
Proofs

WEST: Specification-Based Test Generation for WebAssembly

Dongjun Youn (f52985@kaist.ac.kr)
Wonho Shin (new170527@kaist.ac.kr)
Sukyoung Ryu (sryu.cs@kaist.ac.kr)

Wed 19 Nov, ASE 2025

ASE 2025

KAIST

Wasm Testing

- Testing: Ensures conformance between implementations and the specification
  (Compilers, Interpreters, ...)

```
(assert_return (invoke "mul" (i32.const 1) (i32.const 1)) (i32.const 1))
(assert_return (invoke "mul" (i32.const 1) (i32.const 0)) (i32.const 0))
(assert_return (invoke "mul" (i32.const -1) (i32.const -1)) (i32.const 1))
(assert_trap (invoke "div_s" (i32.const 1) (i32.const 0)) "integer divide by zero")
(assert_trap (invoke "div_s" (i32.const 0) (i32.const 0)) "integer divide by zero")
```

<i32.wast>

Wasm Is Growing!

Wasm 1.0

- Numeric instructions
- Local / global / memory instructions
- Control instructions (block, branch, call)
- ...

https://webassembly.github.io/spec/core/appendix/changes.html

Wasm Is Growing!

Wasm 2.0

- ...
- SIMD
- References
- Table instructions
- Bulk memory / table instructions
- Multi-value
- ...

https://webassembly.github.io/memory64/core/appendix/changes.html

Wasm Is Growing!

Wasm 3.0

- ...
- Garbage collection
- Exception handling
- Tail call
- Relaxed SIMD
- Multi-memory
- 64-bit address
- Typeful references
- ...

https://nvd.nist.gov/vuln/detail/CVE-2024-9122
https://issues.chromium.org/issues/365802567

Wasm Testing

Limitations

- As Wasm grows, manually maintaining tests becomes nontrivial
- Especially, interactions b/w features increase combinatorial complexity

CVE-2024-9122 Detail

Description

Type Confusion in V8 in Google Chrome prior to 129.0.6668.70 allowed a remote attacker to perform out of bounds memory access via a crafted HTML page. (Chromium security severity: High)

VULNERABILITY DETAILS

Summary

Exception Handling + Garbage Collection

WASM type confusion due to imported tag signature subtyping. The imported tag signature is allowed to be a subtype of the defined tag signature, which should instead be invariant.

https://webassembly.org/features/

Wasm Testing

Limitations

- Each Wasm implementation supports different features

	Your browser	Chrome	Firefox	Safari	Node.js	Deno	GraalWasm	Chicory	Wasmtime	Wasmer	wasm2c
Phase 5 - The Feature is Standardized											
JS BigInt to Wasm i64 Integration	✓	85	78	15 [h]	15.0	1.1.2	21.3	N/A	N/A	N/A	N/A
Branch Hinting	?	137	[d]	16	[l]	[t]	✗	✗	✗	✗	✗
Bulk Memory Operations	✓	75	79	15	12.5	0.4	23.0	1.0.0	0.20	1.0	1.0.30
Custom Text Format Annotations	?	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✓	✓	N/A
Extended Constant Expressions	✓	114	112	17.4	21.0	1.33	[aa]	N/A	25	✗	[am]
Garbage Collection	✓	119	120	18.2	22.0	1.38	✗	✗	[ag]	✗	✗

ASE 2025

KAIST

How can we generate Wasm tests for **evolving** specification and **diverging** implementations?

Breakthrough: SpecTec

Mechanized Specification Framework for Wasm

- Official tool for handling the specification itself as data (in OCaml)
- DSL can concisely express syntax, validation rules, and execution semantics

```
syntax instr = …
  | UNOP numtype unop_(numtype)
  | BINOP numtype binop_(numtype)
  | TESTOP numtype testop_(numtype)
  | RELOP numtype relop_(numtype)
  | ...
```

<Sytax>

```
rule Instr_ok/relop:
  C |- RELOP nt relop_nt : nt nt -> I32
```

<Relation: Validation Rule>

```
def $relop(inn, EQ, iN_1, iN_2) =
  $ieq($size(inn), iN_1, iN_2)
def $relop(inn, NE, iN_1, iN_2) =
  $ine($size(inn), iN_1, iN_2)
```

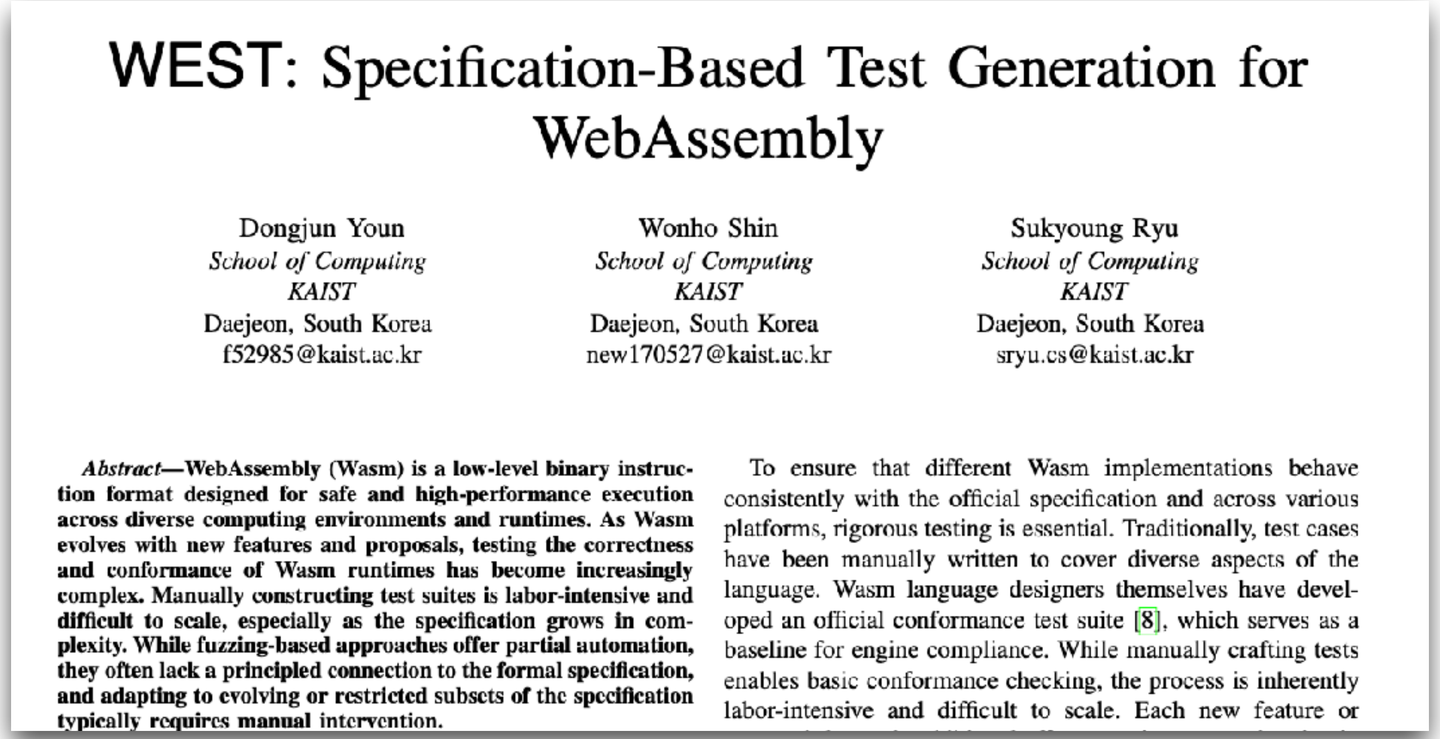
<Auxiliary Function>

```
rule Step_pure/relop:
  (CONST nt c_1) (CONST nt c_2) (RELOP nt relop)
~>(CONST I32 c)
  -- if c = $relop(nt, relop, c_1, c_2)
```

<Relation: Execution Semantics>

WEST

- **We**bAssembly **S**pecification-Based **T**est Generation



WEST: Specification-Based Test Generation for WebAssembly

Dongjun Youn
School of Computing
KAIST
Daejeon, South Korea
f52985@kaist.ac.kr

Wonho Shin
School of Computing
KAIST
Daejeon, South Korea
new170527@kaist.ac.kr

Sukyoung Ryu
School of Computing
KAIST
Daejeon, South Korea
sryu.cs@kaist.ac.kr

***Abstract*—WebAssembly (Wasm) is a low-level binary instruction format designed for safe and high-performance execution across diverse computing environments and runtimes. As Wasm evolves with new features and proposals, testing the correctness and conformance of Wasm runtimes has become increasingly complex. Manually constructing test suites is labor-intensive and difficult to scale, especially as the specification grows in complexity. While fuzzing-based approaches offer partial automation, they often lack a principled connection to the formal specification, and adapting to evolving or restricted subsets of the specification typically requires manual intervention.**

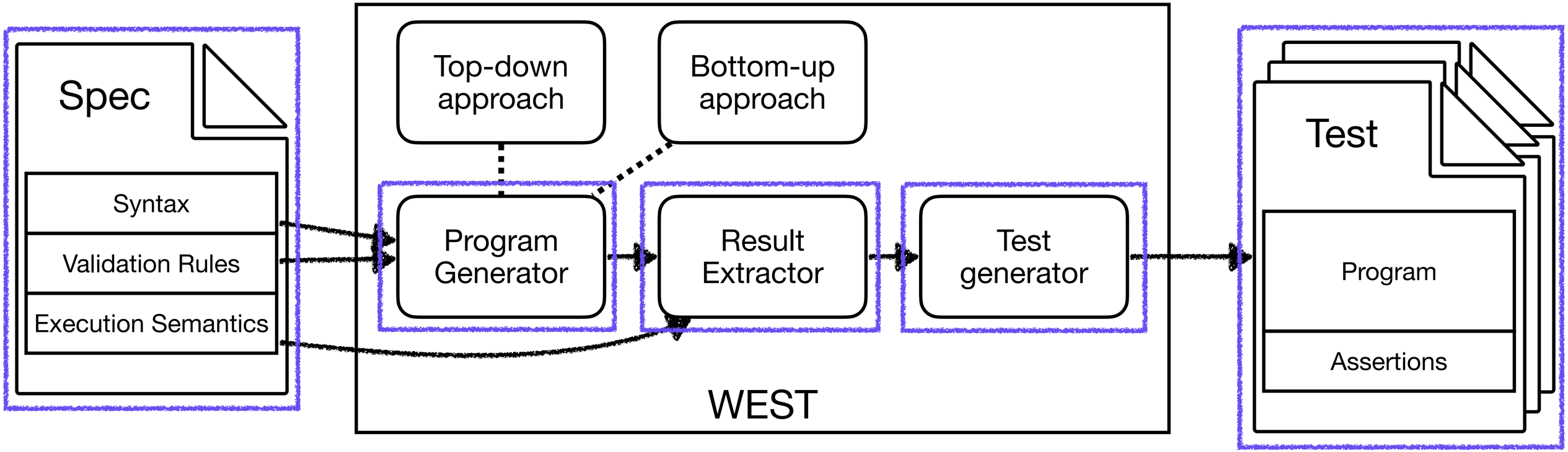
To ensure that different Wasm implementations behave consistently with the official specification and across various platforms, rigorous testing is essential. Traditionally, test cases have been manually written to cover diverse aspects of the language. Wasm language designers themselves have developed an official conformance test suite [8], which serves as a baseline for engine compliance. While manually crafting tests enables basic conformance checking, the process is inherently labor-intensive and difficult to scale. Each new feature or

WEST

Design Goals

- Using SpecTec, automatically generate Wasm tests from a given specification
  - ... including specifications with certain features added or removed
- Generate syntactically correct & valid programs in a systematic manner
  - ... utilizing the mechanized syntax and validation rules expressed in DSL
- Produce semantically rich and diverse tests.
  - ... including various language features and their interactions

Overview


Spec
Syntax
Validation Rules
Execution Semantics
Top-down approach
Bottom-up approach
Program Generator
Result Extractor
Test generator
WEST
Test
Program
Assertions

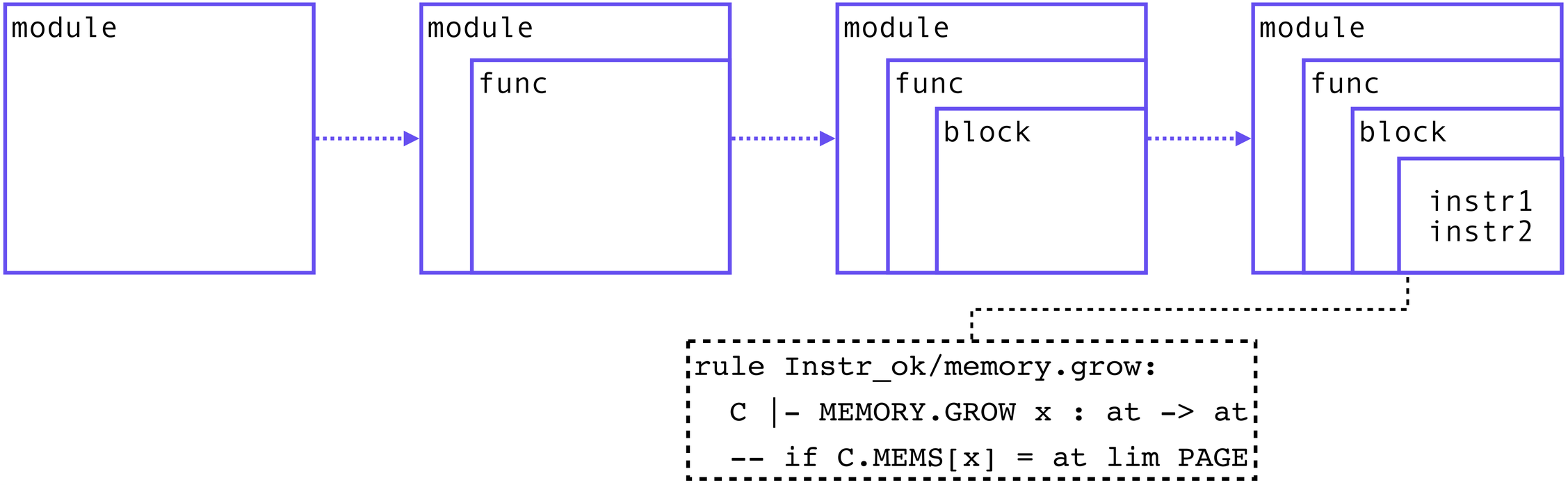
Two Approaches
for Generating Program

- Top-down
  - Starts from the outermost program structure (the top-level module)
  - Specialized for producing exceptional test cases that are often overlooked
- Bottom-up
  - Starts from the innermost program structure (individual instructions)
  - Ensures that specific language features are always included

WEST

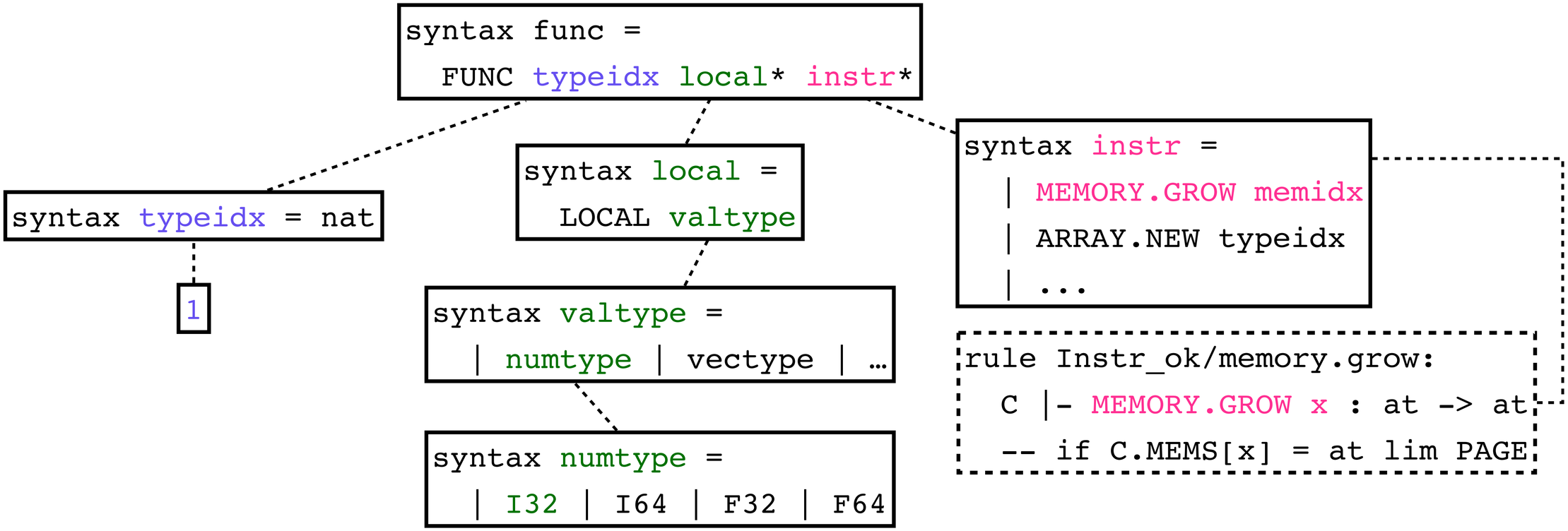
Top-down

- Grammar-based generation that recursively expands non-terminal symbols
- Filter-out invalid immediate structures, using validation rules



WEST

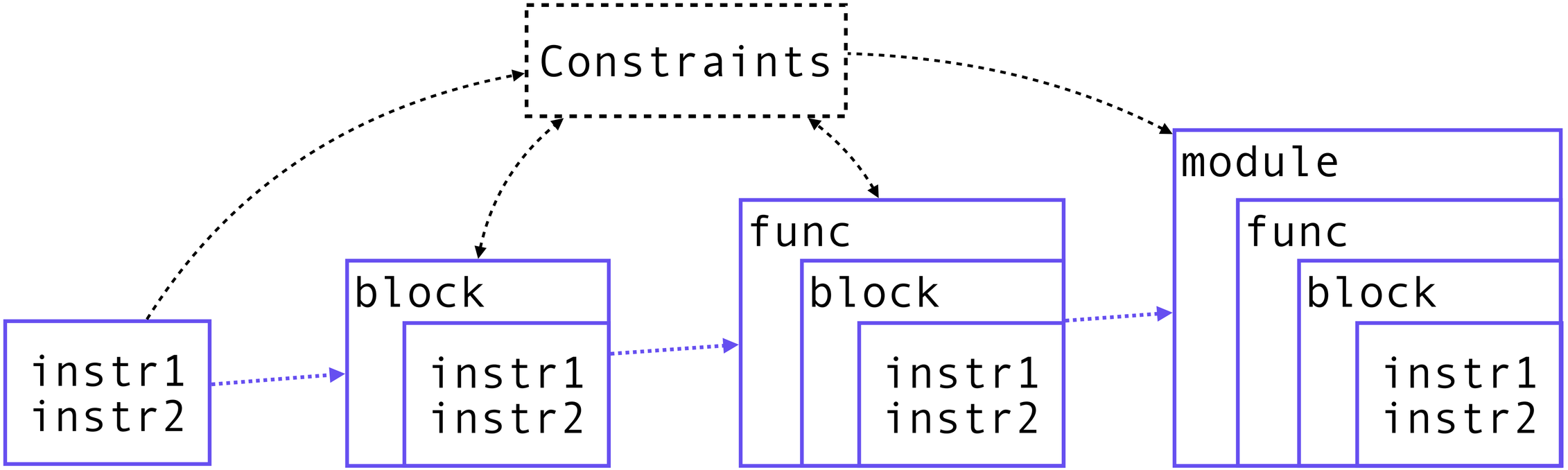
Top-down Example



WEST

Bottom-up

- Given a combination of instructions, generates valid wrappers around them.
- The constraints are gradually collected and used during generation of wrappers.



WEST

Bottom-up Example

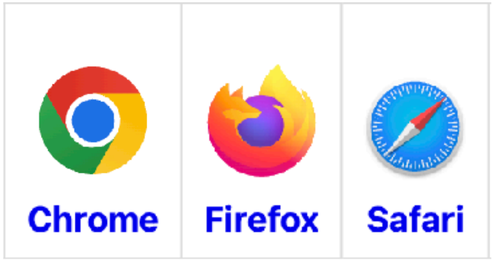
```
rule Instr_ok/memory.grow:
  C |- MEMORY.GROW x : t -> t
  -- if C.MEMS[x] = t lim PAGE

rule Instr_ok/array.new:
  C |- ARRAY.NEW x : I32 -> ...
  -- if ...
```

```
(module
  (type $0 (array (mut i64)))
  (type $1 (func (result (ref 0))))
  (memory $0 i64 1 2)
  (memory $1 i32 1 2)
  (func $0 (type 1)
    i64.const 0
    i32.const 0
    memory.grow 1
    array.new 0
  )
)
```

Evaluation

Bugs Found

- 3 web browser engines (V8 / SpiderMonkey / JavaScriptCore)



- 2 standalone engines (Wasmtime / Wasmer)

- For each engine, use different sub-variants of the specification
  - where unsupported features are removed

Evaluation

Bugs Found

- 16 bugs found, 11 of which are newly found and confirmed

TABLE I: Bugs discovered by generated tests.

Engine	Description	Status
Spider-Monkey	Importing a table with a non-nullable reference type is invalid	Fixed
	`catch_ref` with non-null `exnref` is invalid	Fixed
Java-Script-Core	GC instruction after parametric instruction causes parsing error	Fixed
	`ref.null` after parametric instruction causes parsing error	Fixed
	`try_table` after parametric instruction causes parsing error	Fixed
	Accessing `v128` from null struct/array causes runtime crash†	Fixed
	`throw_ref` with non-nullable reference is invalid	Fixed
	Large `i31ref` with `v128` parameter gives incorrect result	Fixed
	`v128` with block and `throw_ref` causes runtime crash	Dup.
	Importing table with non-nullable type causes parsing error	Fixed
	Importing `exnref` global causes runtime error	Report
Wasm-time	`v128` array as table initializer causes runtime crash*	Dup.
	Out-of-bounds access on `none` reference table does not trap*	Fixed
	`global`/`table.set` with large `i31ref` causes runtime crash*	Dup.
	Exporting function with long result type causes runtime crash	Fixed
Wasmer	An empty data section causes parsing error	Dup.

https://www.w3.org/TR/wasm-js-api-2/

WebAssembly JavaScript Interface

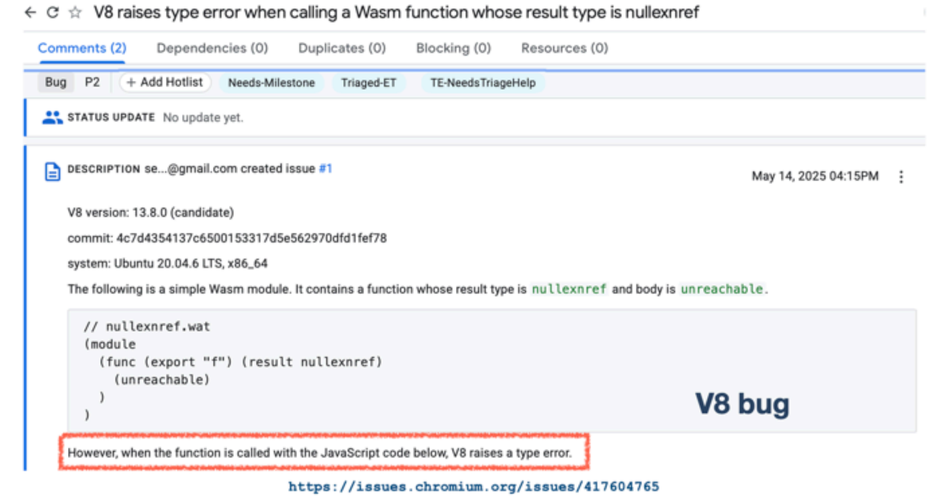
Editor's Draft, 12 August 2026

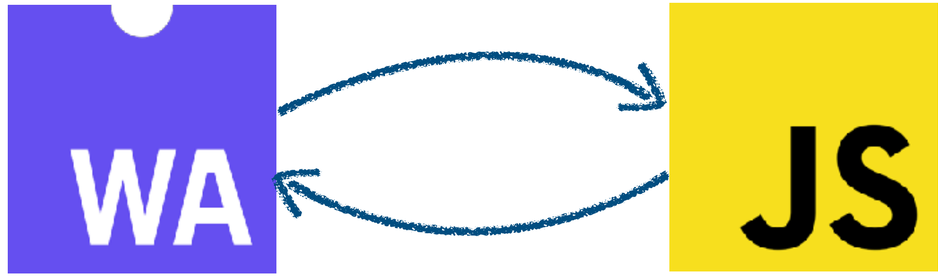
▼ More details about this document

This document provides an explicit JavaScript API for interacting with WebAssembly.

This is part of a collection of related documents: the Core WebAssembly Specification, the WebAssembly JS Interface, and the WebAssembly Web API.

Interoperations with Different Languages





Wasm-SpecTec

ESMeta

https://dl.acm.org/doi/10.1145/2970276.2970368

HybriDroid: Static Analysis Framework for Android Hybrid Applications

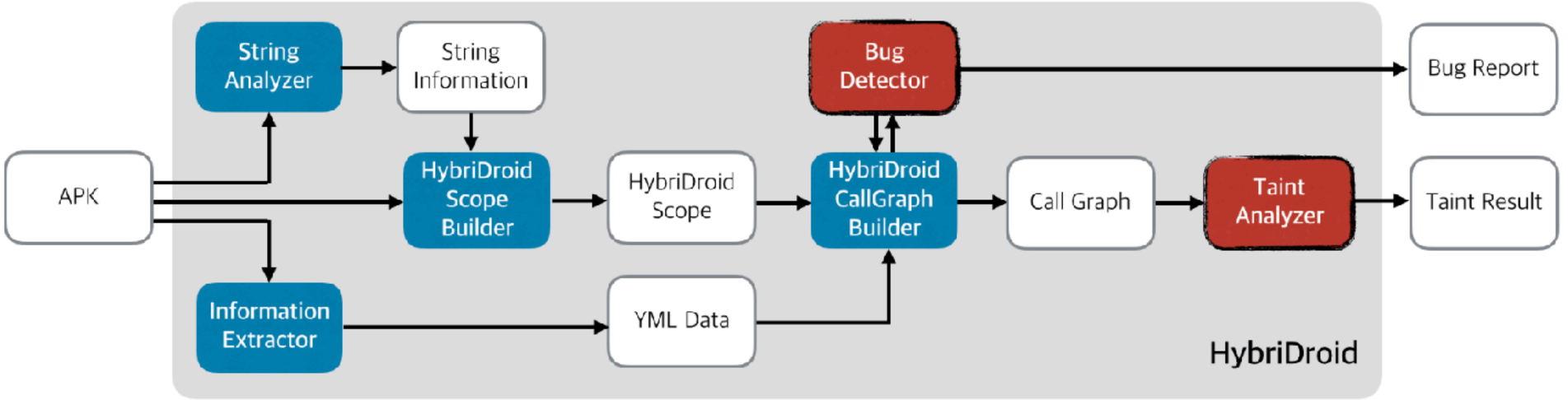


Table 3: Bug detection results

Rank	Hybrid App	Bug Type (#)	#FP	#TP	Bug Cause (#)	Time
1 – 100	com.gameloft.android.ANMP.GloftDMHM	MethodNotFound (1)	0	1	Obfuscation (1)	2404 sec.
	com.creativemobile.DragRacing	MethodNotFound (1)	1	0		3192 sec.
	com.gau.go.launcherex	MethodNotFound (2)	2	0		5432 sec.
	com.tripadvisor.tripadvisor	MethodNotFound (1)	0	1	Obfuscation (1)	4028 sec.
	com.dianxinos.dxbs	MethodNotFound (1)	0	1	Obfuscation (1)	1924 sec.
10,000 – 10,100	com.magmamobile.game.LostWords	MethodNotFound (1)	1	0		475 sec.
20,000 – 20,100	com.daishin	MethodNotFound (1)	0	1	Undeclared Method (1)	6572 sec.
100,000 – 100,100	com.carezone.caredroid.careapp	MethodNotFound (5)	0	5	Missing Annotation (5)	2357 sec.
	com.pateam.kanomthai	MethodNotFound (2)	0	2	Missing Annotation (2)	4209 sec.
	com.acc5.16	MethodNotFound (6)	0	6	Missing Annotation (6)	367 sec.
	jp.cleanup.android	MethodNotFound (1)	1	0		253 sec.
	ligamexicana.futbol	MethodNotFound (2)	2	0		253 sec.
200,000 – 200,100	com.sysapk.weighter	MethodNotFound (1)	0	1	Missing Annotation (1)	106 sec.
	com.youmustescape3guide.free	MethodNotFound (6)	0	6	Missing Annotation (6)	445 sec.
Total		MethodNotFound (31)	7	24	Missing Annotation (20) Obfuscation (3) Undeclared Method (1)	2287 sec.

https://dl.acm.org/doi/10.1109/ICSE.2019.00038

Towards Understanding and Reasoning about Android Interoperations

$$
\begin{array}{lcl}
\mathbf{p} & ::= & \overline{\mathbf{od}}\ \overline{\mathbf{e}} \\
\mathbf{od} & ::= & \texttt{object}\ \mathbf{A}\ \{\ \overline{\mathbf{md}}\ \} \\
\mathbf{md} & ::= & \tau^v\ \mathbf{m}(\overline{\tau^v\ \mathbf{x}})\ \{\mathbf{e}\} \\
\mathbf{e} & ::= & \mathbf{c} \mid \mathbf{x} \mid \mathbf{e}.\mathbf{m}(\overline{\mathbf{e}}) \\
\mathbf{c} & ::= & \mathbf{n} \mid [\mathbf{A}] \mid \texttt{null} \\
\mathbf{v} & ::= & \mathbf{c} \mid () \\
\tau^v & ::= & \iota \mid \mathbf{A} \mid \texttt{void} \\
\mathbf{E} & ::= & []_V \mid \mathbf{E}.\mathbf{m}(\overline{\mathbf{e}}) \mid \mathbf{v}.\mathbf{m}(\overline{\mathbf{v}}\ \mathbf{E}\ \overline{\mathbf{e}})
\end{array}
$$

$$
\text{[Jv-Call]}\quad \frac{\texttt{object}\ \mathbf{A}\ \{\ \overline{\mathbf{md}}\ \} \qquad \tau^v\ \mathbf{m}(\overline{\tau^v\ \mathbf{x}})\ \{\mathbf{e}\} \in \{\ \overline{\mathbf{md}}\ \} \qquad |\overline{\mathbf{x}}| = |\overline{\mathbf{v}}| \qquad \texttt{MatchType}(\mathbf{v}, \tau^v)}{\mathcal{E}[[\mathbf{A}].\mathbf{m}(\overline{\mathbf{v}})]_V \rightarrow \mathcal{E}[\mathbf{e}[\overline{\mathbf{v}/\mathbf{x}}]]}
$$

(a) Java model

$$
\begin{array}{lcl}
\mathbf{e} & ::= & \mathbf{c} \mid \mathbf{x} \mid \texttt{let}\ \mathbf{x} = \mathbf{e}\ \texttt{in}\ \mathbf{e} \mid \mathbf{e}.\mathbf{m} \mid \mathbf{e}.\mathbf{m} = \mathbf{e} \mid \mathbf{e}.\mathbf{m}(\overline{\mathbf{e}}) \\
\mathbf{c} & ::= & \mathbf{n} \mid [\mathbf{A}] \mid \texttt{function}\ (\overline{\mathbf{x}})\ \{\mathbf{e}\} \mid \texttt{null} \\
\mathbf{v} & ::= & \mathbf{c} \mid \texttt{undefined} \\
\mathbf{E} & ::= & []_S \mid \texttt{let}\ \mathbf{x} = \mathbf{E}\ \texttt{in}\ \mathbf{e} \mid \mathbf{E}.\mathbf{m} \mid \mathbf{E}.\mathbf{m} = \mathbf{e} \mid \mathbf{v}.\mathbf{m} = \mathbf{E} \\
 & \mid & \mathbf{E}.\mathbf{m}(\overline{\mathbf{e}}) \mid \mathbf{v}.\mathbf{m}(\overline{\mathbf{v}}\ \mathbf{E}\ \overline{\mathbf{e}})
\end{array}
$$

$$
\text{[Js-Let]}\quad \mathcal{O}\ \mathcal{E}[\texttt{let}\ \mathbf{x} = \mathbf{v}\ \texttt{in}\ \mathbf{e}]_S \rightarrow \mathcal{O}\ \mathcal{E}[\mathbf{e}[\mathbf{v}/\mathbf{x}]]
$$

$$
\text{[Js-Load1]}\quad \frac{\mathcal{O}(\mathbf{A}.\mathbf{m}) = \mathbf{v}}{\mathcal{O}\ \mathcal{E}[[\mathbf{A}].\mathbf{m}]_S \rightarrow \mathcal{O}\ \mathcal{E}[\mathbf{v}]}
$$

$$
\text{[Js-Load2]}\quad \frac{\mathbf{A}.\mathbf{m} \notin \mathit{dom}(\mathcal{O})}{\mathcal{O}\ \mathcal{E}[[\mathbf{A}].\mathbf{m}]_S \rightarrow \mathcal{O}\ \mathcal{E}[\texttt{undefined}]}
$$

$$
\text{[Js-Store]}\quad \mathcal{O}\ \mathcal{E}[[\mathbf{A}].\mathbf{m} = \mathbf{v}]_S \rightarrow \mathcal{O}[\mathbf{A}.\mathbf{m} := \mathbf{v}]\ \mathcal{E}[\mathbf{v}]
$$

$$
\text{[Js-Call]}\quad \frac{\mathcal{O}(\mathbf{A}.\mathbf{m}) = \texttt{function}\ (\overline{\mathbf{x}})\ \{\mathbf{e}\}}{\mathcal{O}\ \mathcal{E}[[\mathbf{A}].\mathbf{m}(\overline{\mathbf{v}})]_S \rightarrow \mathcal{O}\ \mathcal{E}[\mathbf{e}[\overline{\mathbf{v}/\mathbf{x}}]]}
$$

(b) JavaScript model

Fig. 2: Models for Java and JavaScript

$$
\text{[Jv-Add]}\quad \frac{\mathcal{B}' = \mathcal{B} \cup \{([\mathbf{A}], [\mathbf{A}])\}}{\mathcal{B}\ \mathcal{O}\ \mathcal{E}[\texttt{add}([\mathbf{A}])]_V \rightarrow \mathcal{B}'\ \mathcal{O}\ \mathcal{E}[()]}
$$

$$
\text{[Jv-Remove]}\quad \frac{\mathcal{B}' = \mathcal{B} \backslash \{([\mathbf{A}], [\mathbf{A}])\}}{\mathcal{B}\ \mathcal{O}\ \mathcal{E}[\texttt{remove}([\mathbf{A}])]_V \rightarrow \mathcal{B}'\ \mathcal{O}\ \mathcal{E}[()]}
$$

$$
\text{[Jv-Evaluate]}\quad \frac{\mathcal{B} = \overline{([\mathbf{A}], [\mathbf{A}])} \qquad \texttt{object}\ \mathbf{A}\ \{\ \overline{\tau^v\ \mathbf{m}(\overline{\tau^v\ \mathbf{x}})\ \{\mathbf{e}\}}\ \} \qquad \mathcal{O}' = \overline{(\mathbf{A}.\mathbf{m}, \texttt{bridge}\ \mathbf{m})}}{\mathcal{B}\ \mathcal{O}\ \mathcal{E}[\texttt{evaluate}(\mathbf{e})]_V \rightarrow \mathcal{B}\ \mathcal{O}'\ \mathcal{E}[{}^{\texttt{void}}\texttt{VS}(\mathbf{e})]}
$$

$$
\text{[Jv-VS-Void]}\quad \mathcal{B}\ \mathcal{O}\ \mathcal{E}[{}^{\texttt{void}}\texttt{VS}(\mathbf{v})]_V \rightarrow \mathcal{B}\ \mathcal{O}\ \mathcal{E}[()]
$$

$$
\text{[Jv-VS-Num1]}\quad \mathcal{B}\ \mathcal{O}\ \mathcal{E}[{}^{\iota}\texttt{VS}(\mathbf{n})]_V \rightarrow \mathcal{B}\ \mathcal{O}\ \mathcal{E}[\mathbf{n}]
$$

$$
\text{[Jv-VS-Num2]}\quad \frac{\mathbf{v} \neq \mathbf{n}}{\mathcal{B}\ \mathcal{O}\ \mathcal{E}[{}^{\iota}\texttt{VS}(\mathbf{v})]_V \rightarrow \mathcal{B}\ \mathcal{O}\ \mathcal{E}[\mathbf{0}]}
$$

$$
\text{[Jv-VS-Obj1]}\quad \frac{\mathcal{B}([\mathbf{A}]) = [\mathbf{A}]}{\mathcal{B}\ \mathcal{O}\ \mathcal{E}[{}^{\mathbf{A}}\texttt{VS}([\mathbf{A}])]_V \rightarrow \mathcal{B}\ \mathcal{O}\ \mathcal{E}[[\mathbf{A}]]}
$$

$$
\text{[Jv-VS-Obj2]}\quad \frac{\mathbf{v} \notin \mathit{dom}(\mathcal{B})}{\mathcal{B}\ \mathcal{O}\ \mathcal{E}[{}^{\mathbf{A}}\texttt{VS}(\mathbf{v})]_V \rightarrow \mathcal{B}\ \mathcal{O}\ \mathcal{E}[\texttt{null}]}
$$

$$
\text{[Jv-VS-Obj3]}\quad \frac{\mathcal{B}([\mathbf{A}]) \neq [\mathbf{A}]}{\mathcal{B}\ \mathcal{O}\ \mathcal{E}[{}^{\mathbf{A}}\texttt{VS}([\mathbf{A}])]_V \rightsquigarrow \texttt{error"incompatible object"}}
$$

(a) Extended Java semantics

$$
\text{[Js-JsCall]}\quad \frac{\mathcal{O}(\mathbf{A}.\mathbf{m}) = \texttt{function}\ (\overline{\mathbf{x}})\ \{\mathbf{e}\}}{\mathcal{B}\ \mathcal{O}\ \mathcal{E}[[\mathbf{A}].\mathbf{m}(\mathbf{v})]_S \rightarrow \mathcal{B}\ \mathcal{O}\ \mathcal{E}[\mathbf{e}[\overline{\mathbf{v}/\mathbf{x}}]]}
$$

$$
\text{[Js-JvCall1]}\quad \frac{\mathcal{O}(\mathbf{A}.\mathbf{m}) = \texttt{bridge}\ \mathbf{m}' \qquad \mathcal{B}([\mathbf{A}]) = [\mathbf{A}] \qquad \texttt{object}\ \mathbf{A}\ \{\ \overline{\mathbf{md}}\ \} \qquad \tau^v\ \mathbf{m}'(\overline{\tau^v\ \mathbf{x}})\ \{\mathbf{e}\} \in \{\overline{\mathbf{md}}\} \qquad |\mathbf{x}| = |\overline{\mathbf{v}}|}{\mathcal{B}\ \mathcal{O}\ \mathcal{E}[[\mathbf{A}].\mathbf{m}(\mathbf{v})]_S \rightarrow \mathcal{B}\ \mathcal{O}\ \mathcal{E}[\texttt{SV}^{\tau^v}([\mathbf{A}].\mathbf{m}'(\overline{\tau^v\ \texttt{VS}(\mathbf{v})}))]}
$$

$$
\text{[Js-JvCall2]}\quad \frac{\mathcal{O}(\mathbf{A}.\mathbf{m}) = \texttt{bridge}\ \mathbf{m}' \qquad \mathcal{B}([\mathbf{A}]) = [\mathbf{A}] \qquad \texttt{object}\ \mathbf{A}\ \{\ \overline{\mathbf{md}}\ \} \qquad \tau^v\ \mathbf{m}'(\overline{\tau^v\ \mathbf{x}})\ \{\mathbf{e}\} \notin \{\overline{\mathbf{md}}\} \qquad |\mathbf{x}| = |\overline{\mathbf{v}}|}{\mathcal{B}\ \mathcal{O}\ \mathcal{E}[[\mathbf{A}].\mathbf{m}(\overline{\mathbf{v}})]_S \rightsquigarrow \texttt{error "no bridge method"}}
$$

$$
\text{[Js-JvCall3]}\quad \frac{\mathcal{O}(\mathbf{A}.\mathbf{m}) = \texttt{bridge}\ \mathbf{m}' \qquad [\mathbf{A}] \neq \mathit{dom}(\mathcal{B})}{\mathcal{B}\ \mathcal{O}\ \mathcal{E}[[\mathbf{A}].\mathbf{m}(\overline{\mathbf{v}})]_S \rightsquigarrow \texttt{error "non-bridge object"}}
$$

$$
\text{[Js-SV-Void]}\quad \mathcal{B}\ \mathcal{O}\ \mathcal{E}[\texttt{SV}^{\texttt{void}}(())]_S \rightarrow \mathcal{B}\ \mathcal{O}\ \mathcal{E}[\texttt{undefined}]
$$

$$
\text{[Js-SV-Num]}\quad \mathcal{B}\ \mathcal{O}\ \mathcal{E}[\texttt{SV}^{\iota}(\mathbf{n})]_S \rightarrow \mathcal{B}\ \mathcal{O}\ \mathcal{E}[\mathbf{n}]
$$

$$
\text{[Js-SV-Obj1]}\quad \frac{([\mathbf{A}], [\mathbf{A}]) \in \mathcal{B}}{\mathcal{B}\ \mathcal{O}\ \mathcal{E}[\texttt{SV}^{\mathbf{A}}([\mathbf{A}])]_S \rightarrow \mathcal{B}\ \mathcal{O}\ \mathcal{E}[[\mathbf{A}]]}
$$

$$
\text{[Js-SV-Obj2]}\quad \frac{([\mathbf{A}], [\mathbf{A}]) \notin \mathcal{B} \qquad \texttt{object}\ \mathbf{A}\ \{\ \overline{\tau^v\ \mathbf{m}(\overline{\tau^v\ \mathbf{x}})\ \{\mathbf{c}\}}\ \} \qquad \mathcal{B}' = \mathcal{B} \cup \{([\mathbf{A}], [\mathbf{A}])\} \qquad \mathcal{O}' = \mathcal{O} \cup \{(\mathbf{A}.\mathbf{m}, \texttt{bridge}\ \mathbf{m})\}}{\mathcal{B}\ \mathcal{O}\ \mathcal{E}[\texttt{SV}^{\mathbf{A}}([\mathbf{A}])]_S \rightarrow \mathcal{B}'\ \mathcal{O}'\ \mathcal{E}[[\mathbf{A}]]}
$$

(b) Extended JavaScript semantics

https://onlinelibrary.wiley.com/doi/10.1002/spe.3199

RESEARCH ARTICLE

WILEY

Declarative static analysis for multilingual programs using CodeQL

Dongjun Youn[1] | **Sungho Lee**[2] | **Sukyoung Ryu**[1]

2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)

https://dl.acm.org/doi/10.1145/3324884.3416558

Broadening Horizons of Multilingual Static Analysis: Semantic Summary Extraction from C Code for JNI Program Analysis

Sungho Lee
eshaj@cnu.ac.kr
Chungnam National University
South Korea

Hyogun Lee
aasr4r4@kaist.ac.kr
KAIST
South Korea

Sukyoung Ryu
sryu.cs@kaist.ac.kr
KAIST
South Korea

2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)

https://dl.acm.org/doi/10.1109/ICSE43902.2021.00151

JUSTGen: Effective Test Generation for Unspecified JNI Behaviors on JVMs

Sungjae Hwang
School of Computing
KAIST
Daejeon, South Korea
sjhwang87@kaist.ac.kr

Sungho Lee
Department of Computer Science and Engineering
Chungnam National University
Daejeon, South Korea
eshaj@cnu.ac.kr

Jihoon Kim
School of Computing
KAIST
Daejeon, South Korea
kjh618@kaist.ac.kr

Sukyoung Ryu
School of Computing
KAIST
Daejeon, South Korea
sryu.cs@kaist.ac.kr

https://ieeexplore.ieee.org/document/10035436

IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, VOL. 49, NO. 5, MAY 2023

Static Analysis of JNI Programs via Binary Decompilation

Jihee Park, Sungho Lee, Jaemin Hong, and Sukyoung Ryu

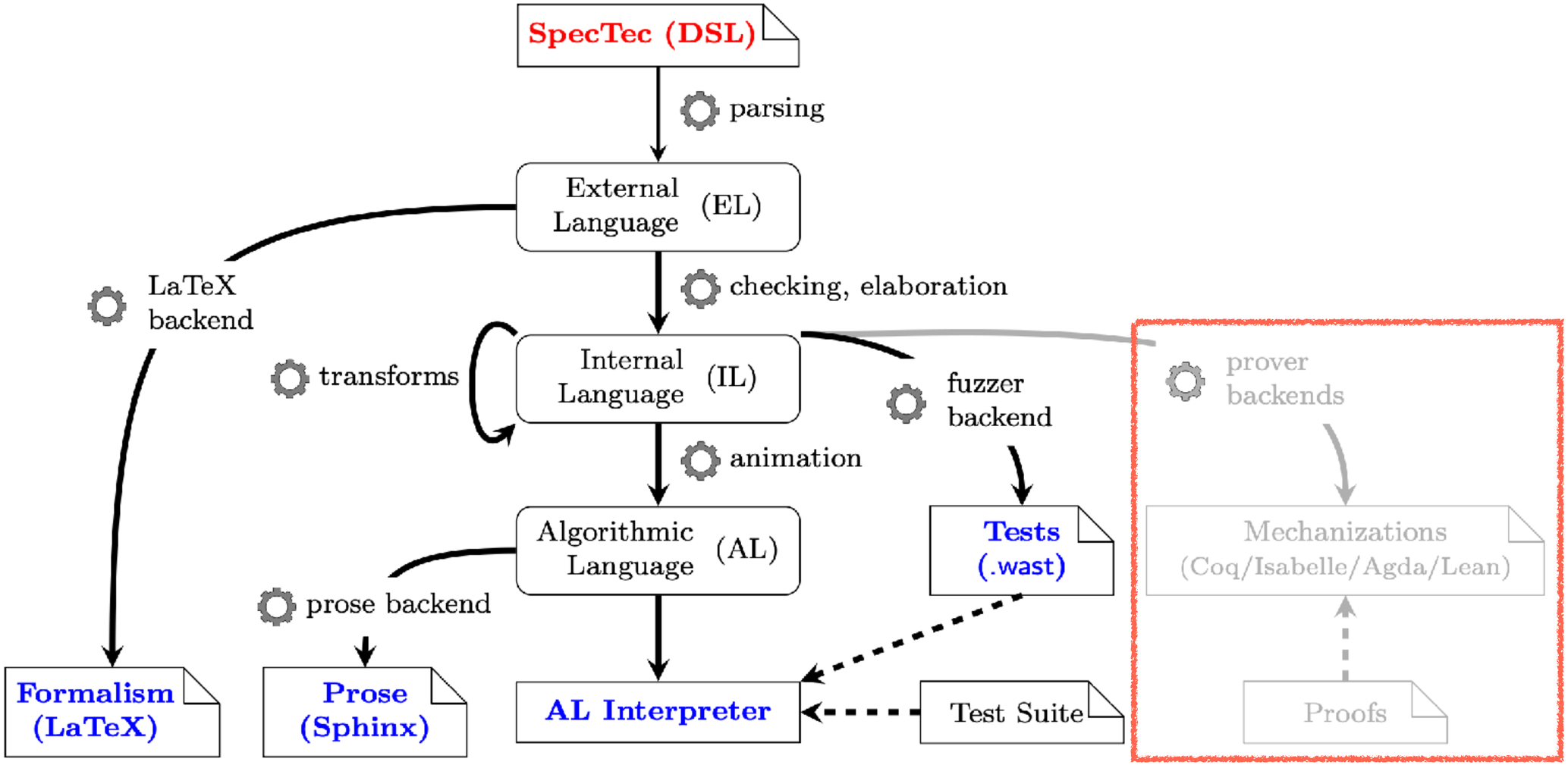
https://ieeexplore.ieee.org/document/10460136

IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, VOL. 50, NO. 4, APRIL 2024

An Empirical Study of JVMs' Behaviors on Erroneous JNI Interoperations

Sungjae Hwang, Sungho Lee, and Sukyoung Ryu

Wasm and SpecTec



Wasm and SpecTec

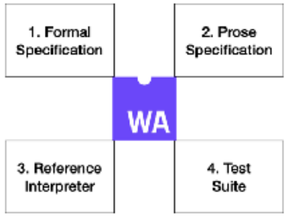
- **March 2023** Dagstuhl seminar
- **October 2023** First presentation @ Wasm CG Meeting
- **June 2024** PLDI paper
- **June 2024** Conditional acceptance of adoption plan @ Wasm CG meeting
- **March 2025** Official acceptance of the adoption plan @ Wasm CG meeting
- **September 2025** Wasm 3.0
- **November 2025** ASE paper (ACM SIGSOFT Distinguished Paper Award)


Suhyeon Ryu


Hoseong Lee


Hyunhee Kang

ESMeta and SpecTec

	Problems	Requirement	Solution	Burden	Community & Researchers
JS	Bugs in the specification and diverse implementations		Type checking of the specification and rich test generation	ESMeta team's maintenance	TC 39 PLRG Programming Language Research Group
Wasm	Bugs and inconsistencies across artifacts		Automatic generation of the artifacts	Community's learning a specification language	WA